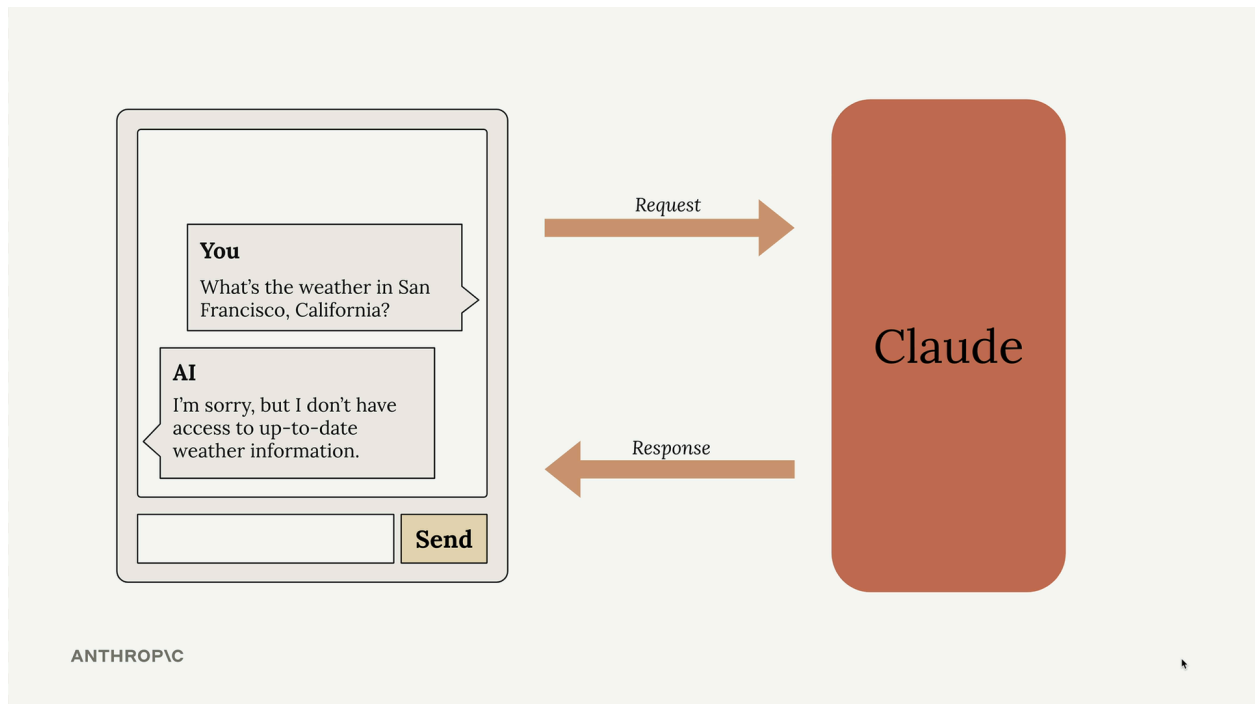


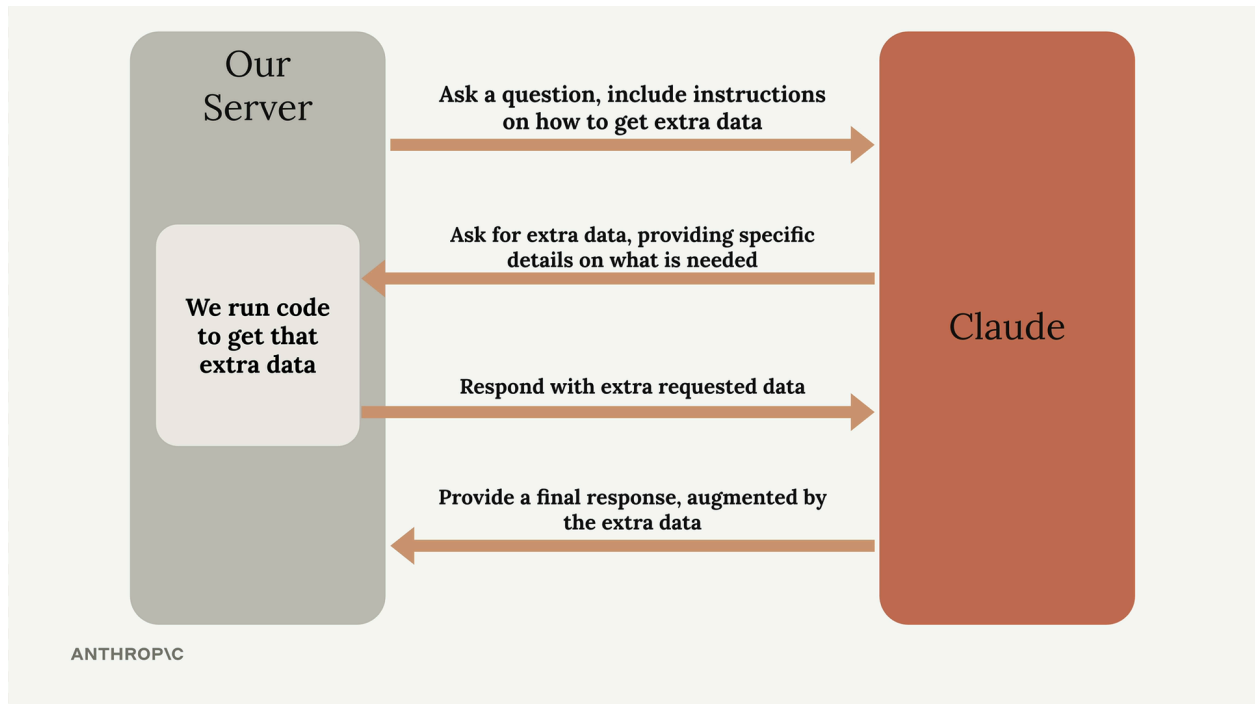
Tools allow Claude to access information from the outside world, solving one of its key limitations. By default, Claude only has access to information it was trained on, which means it can't provide current information like today's weather or recent news.



When a user asks "What's the weather in San Francisco, California?" Claude will typically respond with "I'm sorry, but I don't have access to up-to-date weather information." Tools fix this problem by creating a bridge between Claude and external data sources.

How Tool Use Works

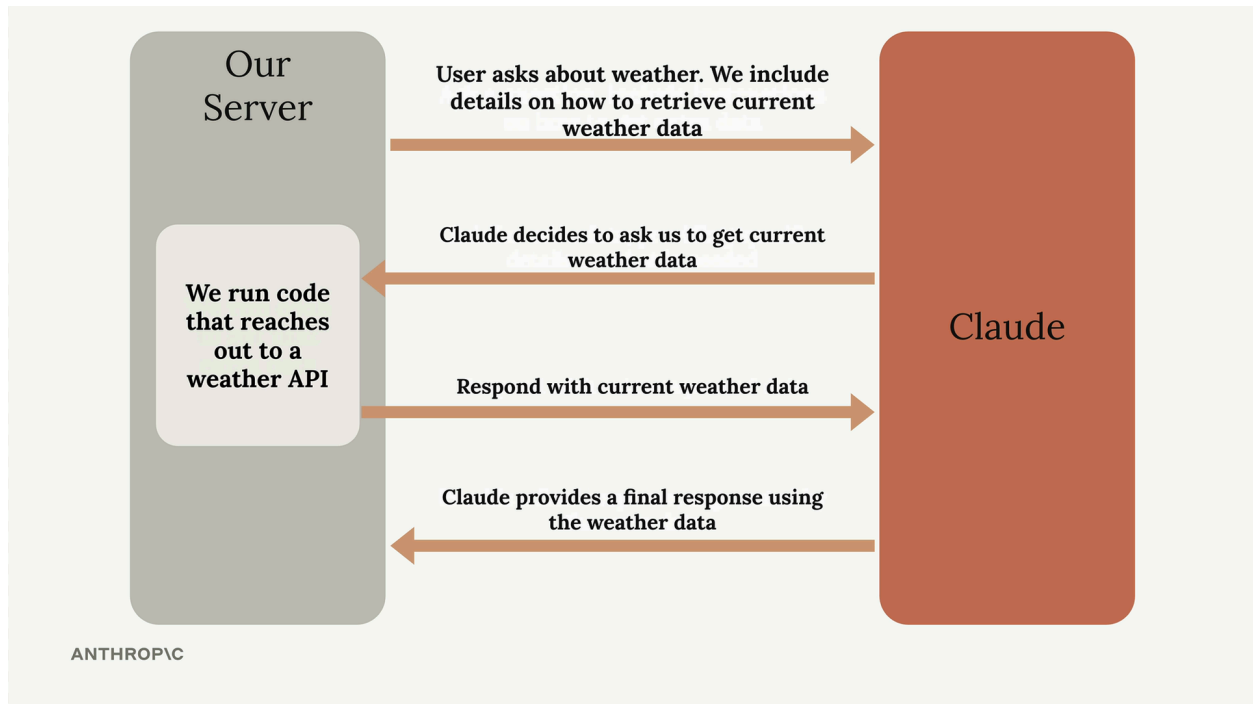
The tool use process follows a specific flow that involves multiple back-and-forth communications between your server and Claude:



1. **Initial Request:** You send Claude a question along with instructions on how to get extra data
2. **Tool Request:** Claude analyzes the question and asks for specific external data it needs
3. **Data Retrieval:** Your server runs code to fetch the requested information
4. **Final Response:** Claude uses the external data to provide a complete, informed answer

Weather Example in Practice

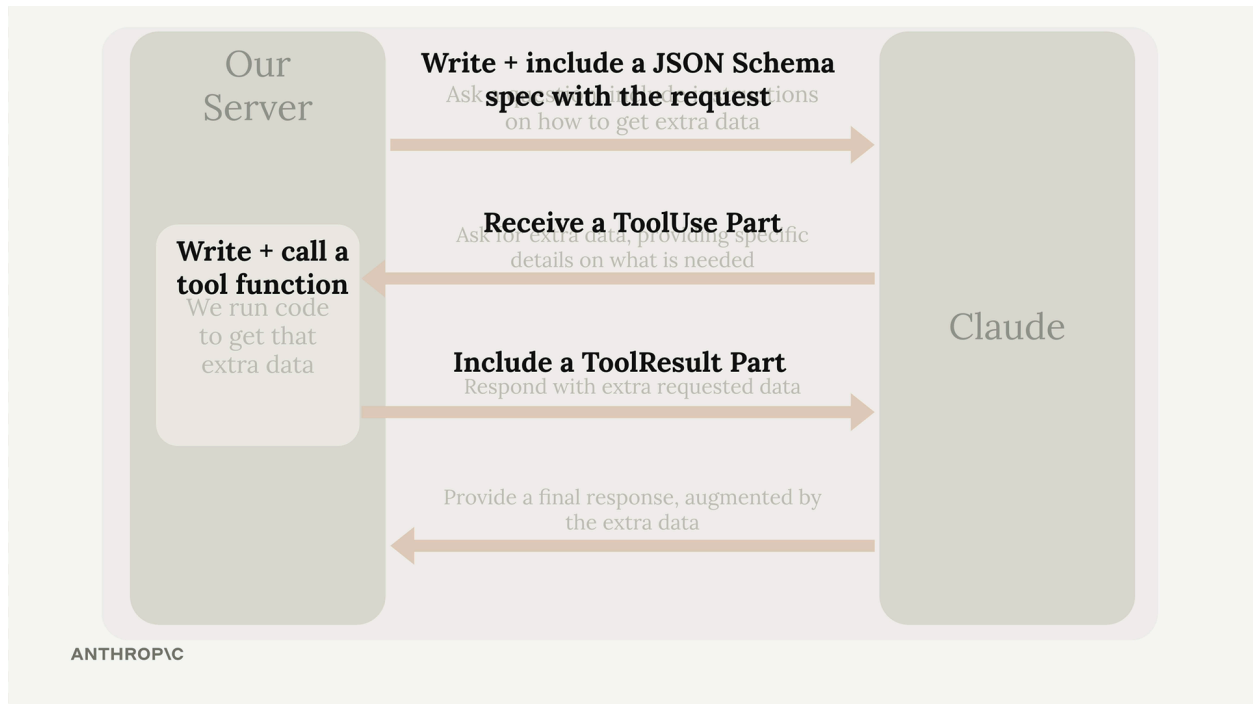
Here's how the tool use flow works for a weather query:



When a user asks about weather, you include details on how to retrieve current weather data in your initial request to Claude. Claude recognizes it needs current weather information and asks your server to get it. Your server calls a weather API, retrieves the live data, and sends it back to Claude. Finally, Claude combines the original question with the fresh weather data to provide an accurate, current response.

Implementation Challenges

Tool use can feel confusing because there's a disconnect between the logical flow and how you actually write the code. The implementation doesn't follow the same order as the conceptual steps:



In practice, you often need to:

- Write the tool function first
- Create a JSON schema specification
- Handle the ToolUse and ToolResult parts
- Include the schema with your request

This jumping around between different parts of the implementation is why tool use initially seems complex. The key is understanding that each step in the logical flow requires specific code components that you'll build in a different order than they execute.

In the following videos, we'll implement tool use step by step, frequently referencing this flow diagram to keep track of which piece we're currently building.

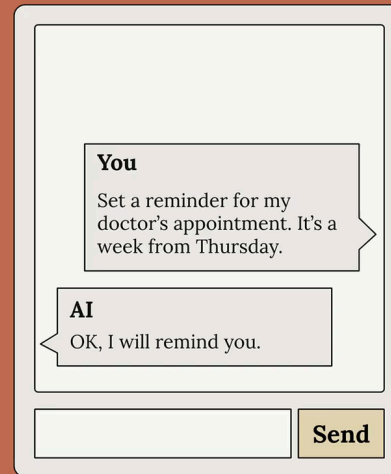
Tool Functions

Building tools for Claude requires solving several challenges that aren't immediately obvious. When you want Claude to set reminders for future dates, you quickly discover that while Claude knows the current date, it doesn't always know the exact time, struggles with complex date arithmetic, and has no built-in way to actually set reminders.

Goal for Tool Use

- We want to teach Claude to set reminders
- Will require us to implement 3-4 custom tools
- For now, we'll focus on just one tool

ANTHROPIC



The solution is to create custom tools that handle these specific tasks. For a reminder system, you'll need three separate tools: one to get the current date and time, another to add durations to dates, and a third to actually set the reminder.

Why This Is Challenging

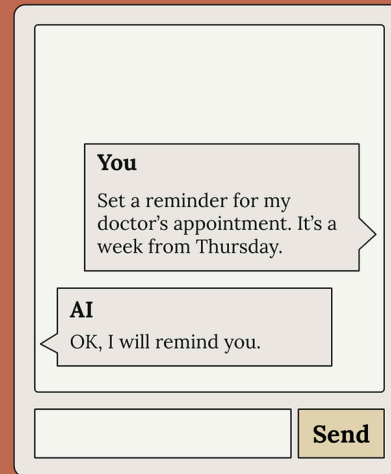
Claude has some limitations when it comes to time-based tasks:

- Claude might know the current date, but not the exact time
- Claude doesn't always handle time-based addition well, especially when looking many days into the future
- Claude doesn't know how to set a reminder

Why is this hard?

- Claude might know the current date, but not the exact time
- Claude doesn't always handle time-based addition well, especially if looking many days into the future
- Claude doesn't know how to set a reminder!

ANTHROPIC



The Tools You Need

To solve these problems, you'll create three dedicated tools:

- **Get the current date time** - Claude needs to know the current date and time
- **Add duration to date time** - Claude isn't perfect with date time addition
- **Set a reminder** - Need a way to set a reminder

Tools We Need

Get the current date time

- Claude needs to know the current date and time

Add duration to date time

- Claude isn't perfect with date time addition

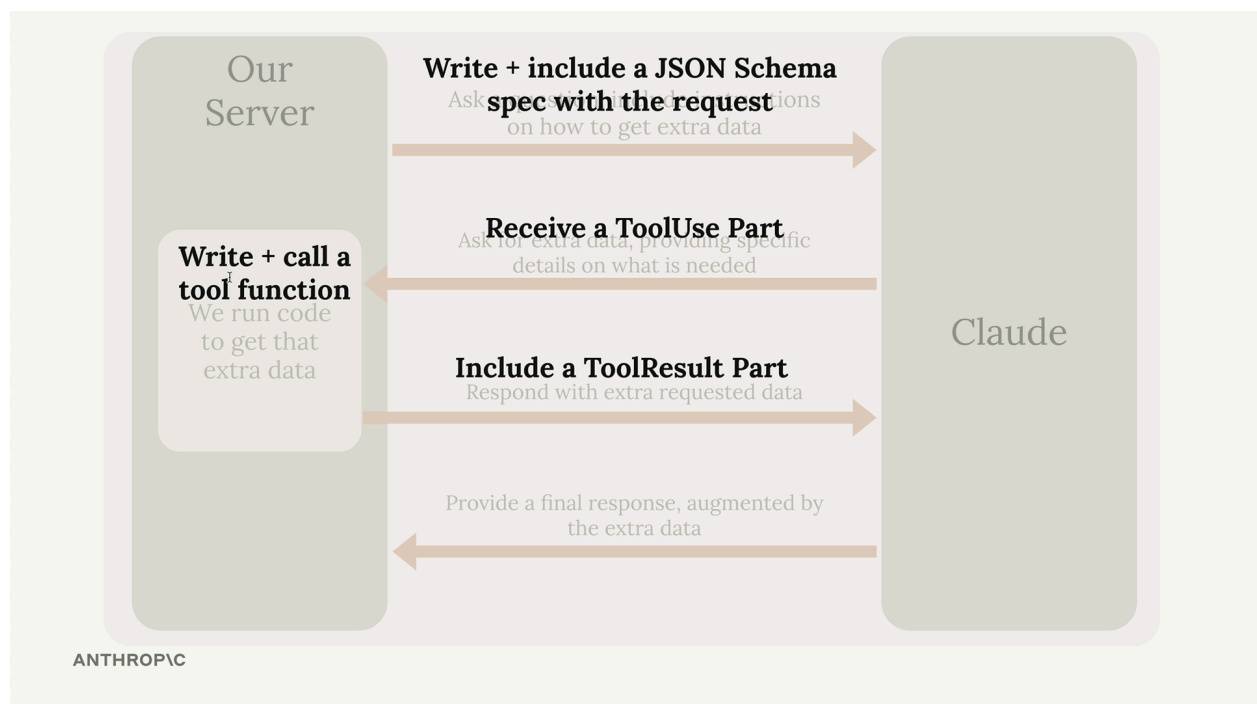
Set a reminder

- Need a way to set a reminder

ANTHROPIC

How Tool Functions Work

The tool system follows a specific flow between your server and Claude. You write functions that Claude can call when it needs additional information, and Claude receives the results to help formulate its response.



The process involves several steps: writing the tool function, creating a JSON schema specification, calling Claude with that schema, running the tool when Claude requests it, and providing the results back to Claude.

Write a tool function

Write a JSON Schema

Call Claude with JSON Schema

Run Tool

Add Tool Result and call Claude again

Tool Function

- Plain Python function that will be executed when Claude decides it needs some additional information to help the user
- **Best practices**
 - Use well-named, descriptive arguments (important later)
 - Validate the inputs, raising an error if they fail validation
 - Return meaningful errors - Claude will try to call to use your function a second time!

```
def get_weather(location, api_key="api_key"):
    if not location or location.strip() == "":
        raise ValueError("Location cannot be empty")

    url = "https://fakeweatherapi.example.com/current"
    params = {
        "q": location,
        "appid": api_key,
        "units": "metric"
    }

    response = requests.get(url, params=params, timeout=10)
    response.raise_for_status()

    return response.json()
```

ANTHROPIC

Writing Tool Functions

Tool functions are plain Python functions that get executed when Claude decides it needs additional information to help the user. Here's how to write them effectively:

Best Practices

- Use well-named, descriptive arguments (this becomes important later)
- Validate the inputs, raising an error if they fail validation
- Return meaningful errors - Claude will try to call your function a second time if it gets an error

```
001_tools.ipynb x
001_tools.ipynb
Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | .venv (Python 3.12)

# Client Setup
import boto3

client = boto3.client("bedrock-runtime", region_name="us-west-2")
model_id = "us.anthropic.claude-3-7-sonnet-20250219-v1:0"

def add_user_message(messages, text):
    user_message = {"role": "user", "content": [{"text": text}]}
    messages.append(user_message)

def add_assistant_message(messages, text):
    assistant_message = {"role": "assistant", "content": [{"text": text}]}
    messages.append(assistant_message)

def chat(messages, system=None, temperature=1.0, stop_sequences=[]):
    params = {
        "modelId": model_id,
        "messages": messages,
        "inferenceConfig": {
```

Creating Your First Tool

Let's start with the simplest tool - getting the current date and time. This function takes a date format parameter and returns the current timestamp:

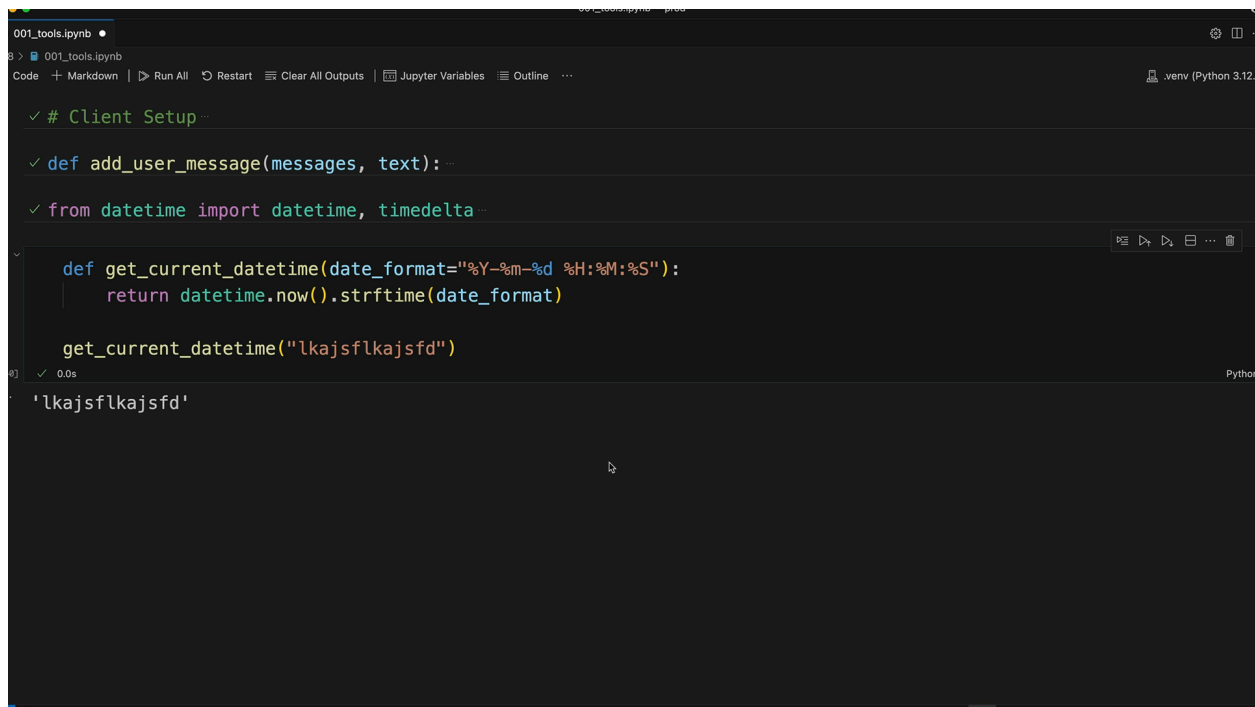
```
from datetime import datetime, timedelta
```

```
def get_current_datetime(date_format="%Y-%m-%d %H:%M:%S"):
    return datetime.now().strftime(date_format)
```

This function is straightforward but follows the key principles: it has a descriptive name, takes a well-named parameter with a sensible default, and returns exactly what it promises.

JSON Schema Specification

Once you have your function, you need to write a JSON Schema that describes it to Claude. This schema tells Claude what arguments the function requires and helps it understand when and how to use the tool.



```
001_tools.ipynb •
001_tools.ipynb
Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline ...
venv (Python 3.12)

✓ # Client Setup...

✓ def add_user_message(messages, text): ...

✓ from datetime import datetime, timedelta ...

def get_current_datetime(date_format="%Y-%m-%d %H:%M:%S"):
    return datetime.now().strftime(date_format)

get_current_datetime("lkajsflkajsfd")

0.0s Python
'lkajsflkajsfd'
```

The JSON Schema serves two purposes: it helps Claude understand what arguments your function requires, and it's not just an LLM concept - JSON Schema is commonly used for data validation across many programming contexts. There are plenty of online tools to help you generate schemas.

Schema Best Practices

- Explain what the tool does, when to use it, and what it returns
- Aim for 3 to 4 sentences in your descriptions
- Provide detailed descriptions for parameters

With your tool function written and schema defined, you're ready to integrate it with Claude and start building more sophisticated AI interactions that can handle real-world tasks like setting reminders.

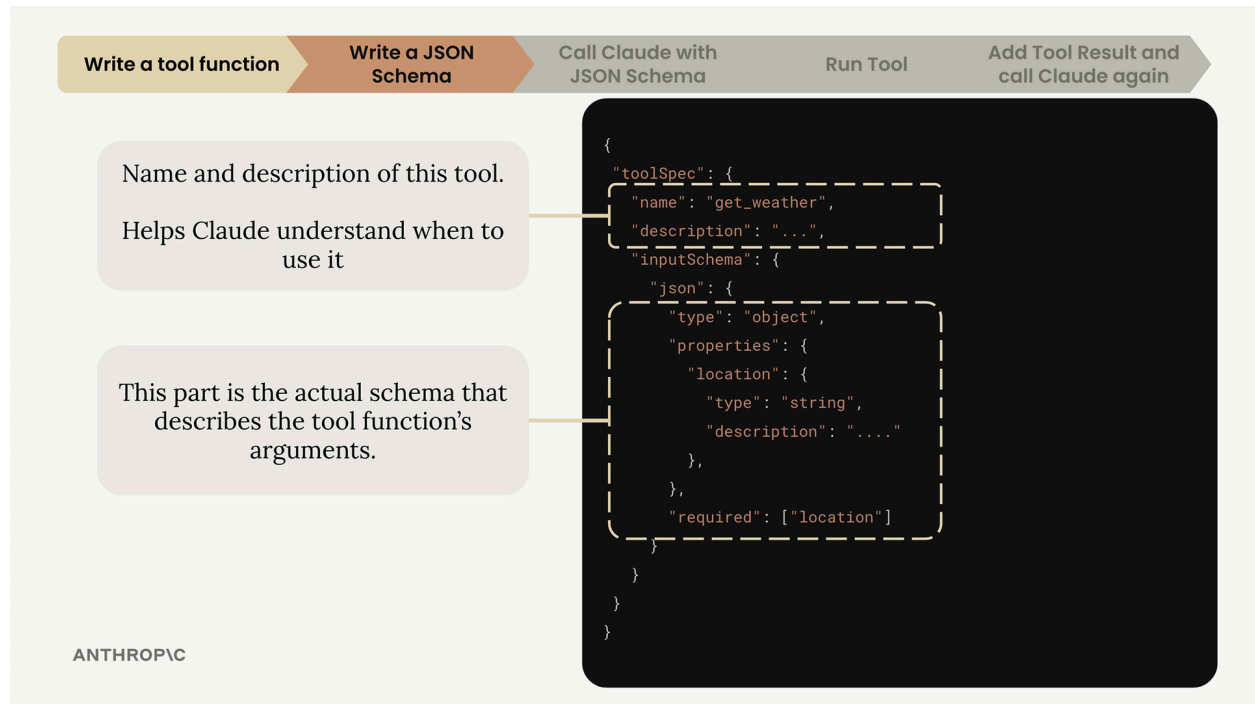
JSON Schema for tools

After creating your tool function, the next step is writing a JSON schema to describe it. This schema tells Claude what arguments your function expects and how to use it properly. While the configuration might look intimidating at first, it's actually straightforward once you understand the process.

Understanding JSON Schema

JSON Schema isn't something invented just for AI tools - it's been around for years as a standard way to validate data. The schema has two main parts: the name and description at the

top (which help Claude understand when to use the tool), and the actual schema that describes the function's arguments.



The top section contains the tool's name and description, which helps Claude understand when to use it. The bottom section is the actual schema that describes your function's arguments in detail.

Creating a JSON Schema: Step-by-Step

Here's the simplest way to create a JSON schema for any function:

Step 1: Write a Dictionary with Sample Data

Take your function and create a dictionary of all keyword arguments with sample data. For example, if you have a function like this:

```
def process_data(ids, profile, primary_id, value):
    pass
```

Create a dictionary with sample values:

Write a tool function

Write a JSON
Schema

Call Claude with
JSON Schema

Run Tool

Add Tool Result and
call Claude again

Step 1: Write a dictionary of keyword args for your function with sample data

```
def process_data(ids, profile, primary_id, value):  
    pass  
  
# Example call:  
process_data(ids=[1,2,3], profile={"active": True}, primary_id="45", value=20)
```

```
{  
  "ids": [1],  
  "profile": {  
    "active": True  
  },  
  "primary_id": "abc",  
  "value": 20  
}
```

ANTHROPIC

Step 2: Convert to JSON

Convert your Python dictionary to proper JSON format. The main difference is changing Python's `True` to JSON's `true`.

Write a tool function

Write a JSON
Schema

Call Claude with
JSON Schema

Run Tool

Add Tool Result and
call Claude again

Step 2: Convert the dictionary to JSON

```
def process_data(ids, profile, primary_id, value):  
    pass  
  
# Example call:  
process_data(ids=[1,2,3], profile={"active": True}, primary_id="45", value=20)
```

```
{  
  "ids": [1],  
  "profile": {  
    "active": true  
  },  
  "primary_id": "abc",  
  "value": 20  
}
```

ANTHROPIC

Step 3: Use an Online Converter

Search for "JSON to JSON Schema converter" and use one of the many free online tools. Paste your JSON data and let it generate the schema automatically.

The screenshot shows the 'Free Online JSON to JSON Schema Converter' interface. At the top, there's a navigation bar with links for 'Products', 'Pricing', 'Download', 'Support', and 'Company'. Below the header, the title 'Free Online JSON to JSON Schema Converter' is displayed. A sub-header states 'Uses the sample JSON document to infer a JSON schema.' The main section is titled 'Sample JSON Document' and contains a text area with the following JSON data:

```
1 {
2   "ids": [1],
3   "profile": {
4     "active": true
5   },
6   "primary_id": "abc",
7   "value": 20
8 }
9
```

Below the text area is an 'Options' dropdown menu and a 'Generate Schema' button. The bottom section, titled 'Inferred JSON Schema', displays the generated schema:

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "type": "object",
  "properties": {
    "ids": {
      "type": "array",
      "items": {

```

The tool will analyze your sample data and create a proper schema structure. Remove any `$schema` declarations from the output - you don't need them.

Step 4: Add Descriptions

The most important step is adding detailed descriptions to each property. These descriptions help Claude understand exactly what each argument does and how to use it.

Write a tool function

Write a JSON Schema

Call Claude with JSON Schema

Run Tool

Add Tool Result and call Claude again

Step 4: Add a 'description' to each property

```
{
  "type": "object",
  "properties": {
    "ids": {
      "type": "array",
      "description": "...",
      "items": { "type": "number" }
    },
    "primary_id": {
      "type": "string",
      "description": "..."
    }
    ...other properties...
  }
}
```

ANTHROPIC

Writing Good Descriptions

When writing descriptions for your tools and properties, follow these best practices:

- Explain what the tool does, when to use it, and what it returns
- Aim for 3-4 sentences in your tool description
- Provide super detailed descriptions for each property
- If you're stuck, paste your function into Claude and ask it to write descriptions for you

Here's an example of a well-described tool schema:

Write a tool function

Write a JSON Schema

Call Claude with JSON Schema

Run Tool

Add Tool Result and call Claude again

Write a JSON Schema spec to describe your function

- The JSON Schema helps Claude understand what arguments your function requires
- JSON Schema is not just a LLM thing - this is a commonly used for data validation
 - There are tons of tools online that can help you generate a JSON Schema
- **Best practices**
 - Explain what the tool does, when to use it, and what it returns
 - Aim for 3 to 4 sentences
 - Provide super detailed descriptions.

ANTHROPIC

```
{
  "toolSpec": {
    "name": "get_weather",
    "description": "Retrieves current weather information for a specified location. Connects to a weather service API and returns basic weather data, specifically temperature, conditions, and humidity. Use to understand current weather conditions. The data is returned in metric units.",
    "inputSchema": {
      "json": {
        "type": "object",
        "properties": {
          "location": {
            "type": "string",
            "description": "The location for which to retrieve weather data. Can be a city name (e.g., 'London'), city and country code (e.g., 'London,UK'), or geographic coordinates. Cannot be empty."
          },
          "required": ["location"]
        }
      }
    }
  }
}
```

Notice how the description clearly explains what the weather tool does, when to use it, what data it returns, and provides specific examples of valid location formats.

Putting It All Together

Your final JSON schema should look something like this structure, with the **toolSpec** containing the name, description, and **inputSchema** with the detailed argument specifications:

Write a tool function

Write a JSON Schema

Call Claude with JSON Schema

Run Tool

Add Tool Result and call Claude again

Write a JSON Schema spec to describe your function

- The JSON Schema helps Claude understand what arguments your function requires
- JSON Schema is not just a LLM thing - this is a commonly used for data validation
 - There are tons of tools online that can help you generate a JSON Schema
- **Best practices**
 - Explain what the tool does, when to use it, and what it returns
 - Aim for 3 to 4 sentences
 - Provide super detailed descriptions.

ANTHROPIC

```
{
  "toolSpec": {
    "name": "get_weather",
    "description": "Retrieves current weather information for a specified location. Connects to a weather service API and returns basic weather data, specifically temperature, conditions, and humidity. Use to understand current weather conditions. The data is returned in metric units.",
    "inputSchema": {
      "json": {
        "type": "object",
        "properties": {
          "location": {
            "type": "string",
            "description": "The location for which to retrieve weather data. Can be a city name (e.g., 'London'), city and country code (e.g., 'London,UK'), or geographic coordinates. Cannot be empty."
          },
          "required": ["location"]
        }
      }
    }
  }
}
```

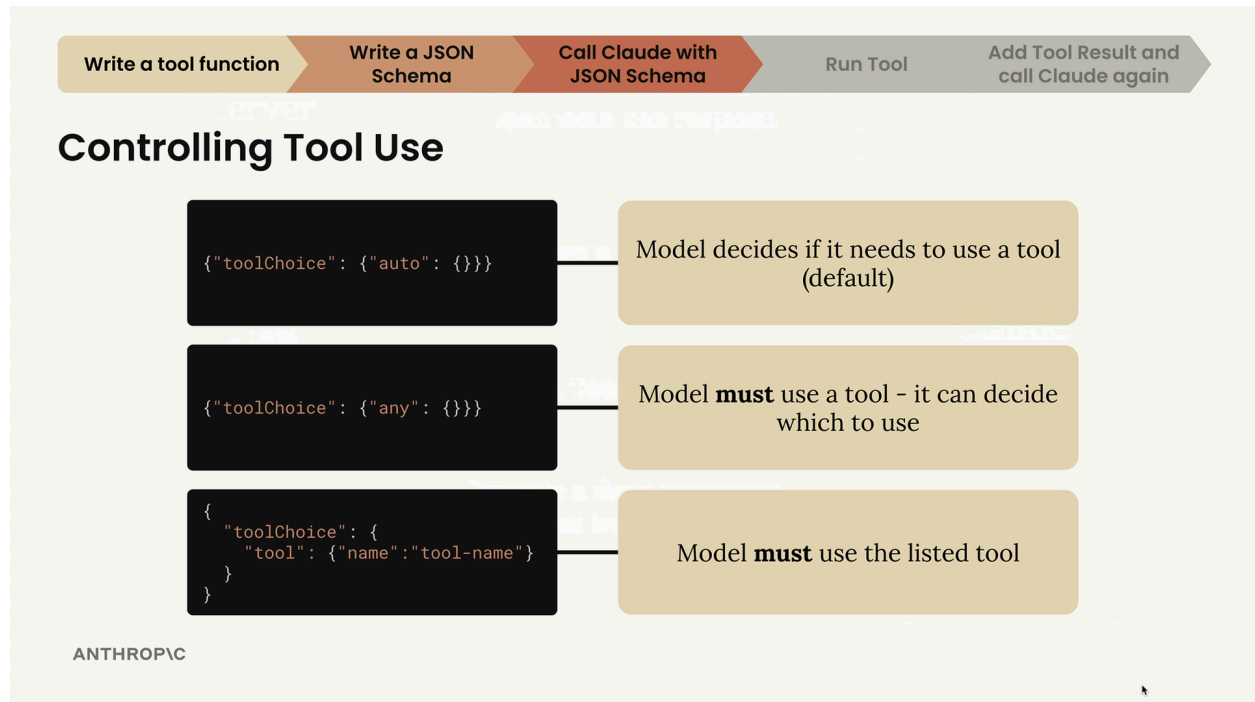
The schema acts as a contract between your code and Claude, ensuring that when Claude decides to use your tool, it knows exactly what information to provide and in what format. This clear communication is what makes tool use reliable and effective.

Handling tool use responses

When Claude decides to use a tool, it returns a special response structure that requires careful handling. Understanding this response format and implementing proper conversation management is crucial for building robust tool-enabled applications.

Tool Choice Configuration

Before diving into responses, it's worth understanding how to control when Claude uses tools. The `toolChoice` parameter gives you three options:



- **auto** - Claude decides whether to use a tool (default behavior)
- **any** - Claude must use a tool but can choose which one
- **specific tool** - Force Claude to use a particular tool by name

The third option is especially useful for testing when you want to ensure Claude calls a specific function.

Multi-Part Message Structure

When Claude wants to use a tool, it returns an assistant message with multiple content parts instead of just text:

Write a tool function

Write a JSON
Schema

Call Claude with
JSON Schema

Run Tool

Add Tool Result and
call Claude again

Multi-Part Messages

Assistant Message's Content List	"Text" Part	<i>"I can help you find out the current time. Let me find that information for you"</i>
	"toolUse" Part	<pre>{ "toolUse": { "toolUseId": "tooluse_w68iAzGq-Hwg", "name": "get_current_datetime", "input": {"date_format": "%H:%M:%S"} } }</pre>

ANTHROPIC

The response contains two parts:

- **Text Part** - Human-readable explanation like "I can help you find out the current time. Let me find that information for you"
- **ToolUse Part** - Structured data telling you which tool to run and with what arguments

Understanding the ToolUse Part

The ToolUse part contains three key pieces of information:

Write a tool function

Write a JSON
Schema

Call Claude with
JSON Schema

Run Tool

Add Tool Result and
call Claude again

Tool Use Part

- **“toolUseId”** - Used when we send back a ToolResult Part - more on this very soon
- **“name”** - Name of the tool that the model wants to call. Matches the “name” in your provided tool spec
- **“input”** - Arguments the model wants to provide to your tool function

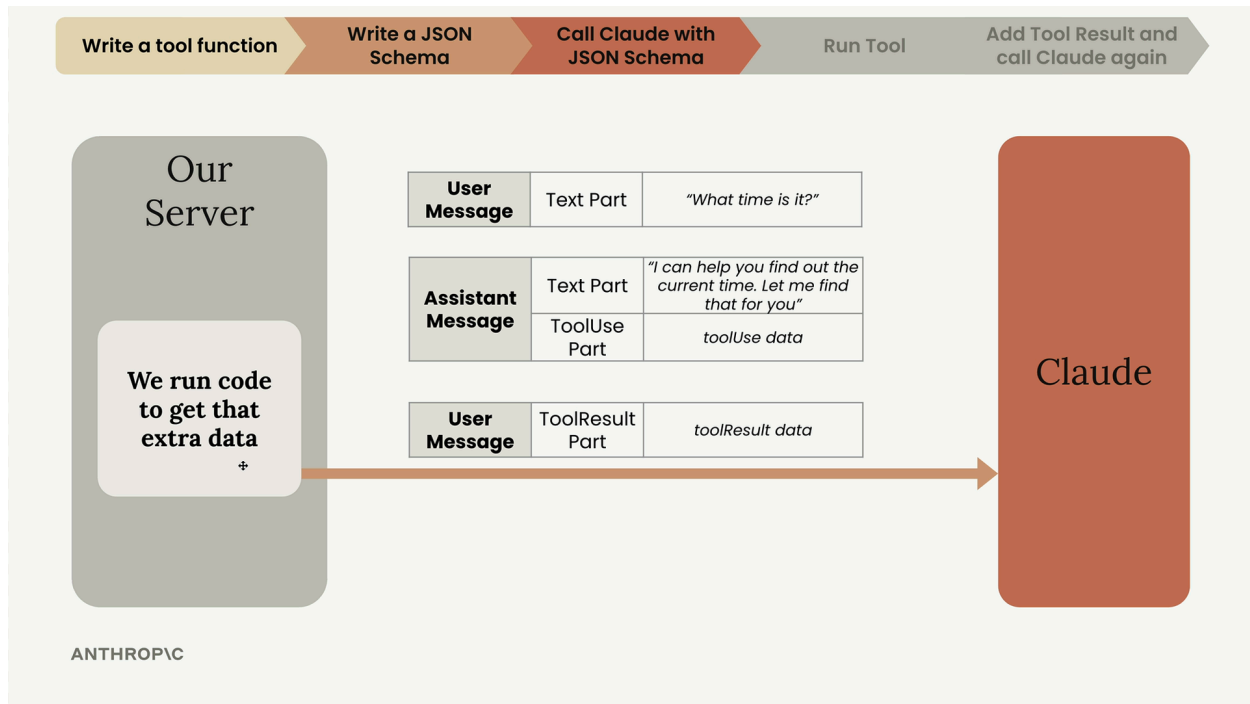
```
{  
  "toolUse": {  
    "toolUseId": "tooluse_w68iAzGq-Hwg",  
    "name": "get_current_datetime",  
    "input": {"date_format": "%H:%M:%S"}  
  }  
}
```

ANTHROPIC

- **toolUseId** - A unique identifier you'll need when sending back the tool result
- **name** - The exact tool name from your JSON schema that Claude wants to call
- **input** - A dictionary of arguments Claude wants to pass to your tool function

Conversation Flow with Tools

Tool usage follows a specific conversation pattern that requires maintaining complete message history:



When you receive a tool use request, you need to:

1. Extract the tool information from the ToolUse part
2. Run your actual tool function
3. Send back a ToolResult message along with the complete conversation history
4. Include the original user message and the assistant's tool use message in your next request

Updating Helper Functions

To handle multi-part messages properly, you'll need to update your message handling functions. Here's how to make your functions flexible enough to handle both simple text and complex multi-part content:

```
def add_user_message(messages, content):
    if isinstance(content, str):
        user_message = {"role": "user", "content": [{"text": content}]}
    else:
        user_message = {"role": "user", "content": content}
    messages.append(user_message)

def add_assistant_message(messages, content):
    if isinstance(content, str):
        assistant_message = {"role": "assistant", "content": [{"text": content}]}
    else:
        assistant_message = {"role": "assistant", "content": content}
    messages.append(assistant_message)
```



```
assistant_message = {"role": "assistant", "content": content}
messages.append(assistant_message)
```

You'll also want to update your chat function to return both the text and the full parts list:

```
def chat(messages, system=None, temperature=1.0, stop_sequences=[], tools=None):
    # ... existing setup code ...

    response = client.converse(**params)

    text = response["output"]["message"]["content"][0]["text"]
    parts = response["output"]["message"]["content"]

    return text, parts
```

Checking the Stop Reason

Always check the `stopReason` field in Claude's response. When it equals `"tool_use"`, you know Claude wants to call a tool rather than just providing a text response. This is your signal to extract the tool information and execute the requested function.

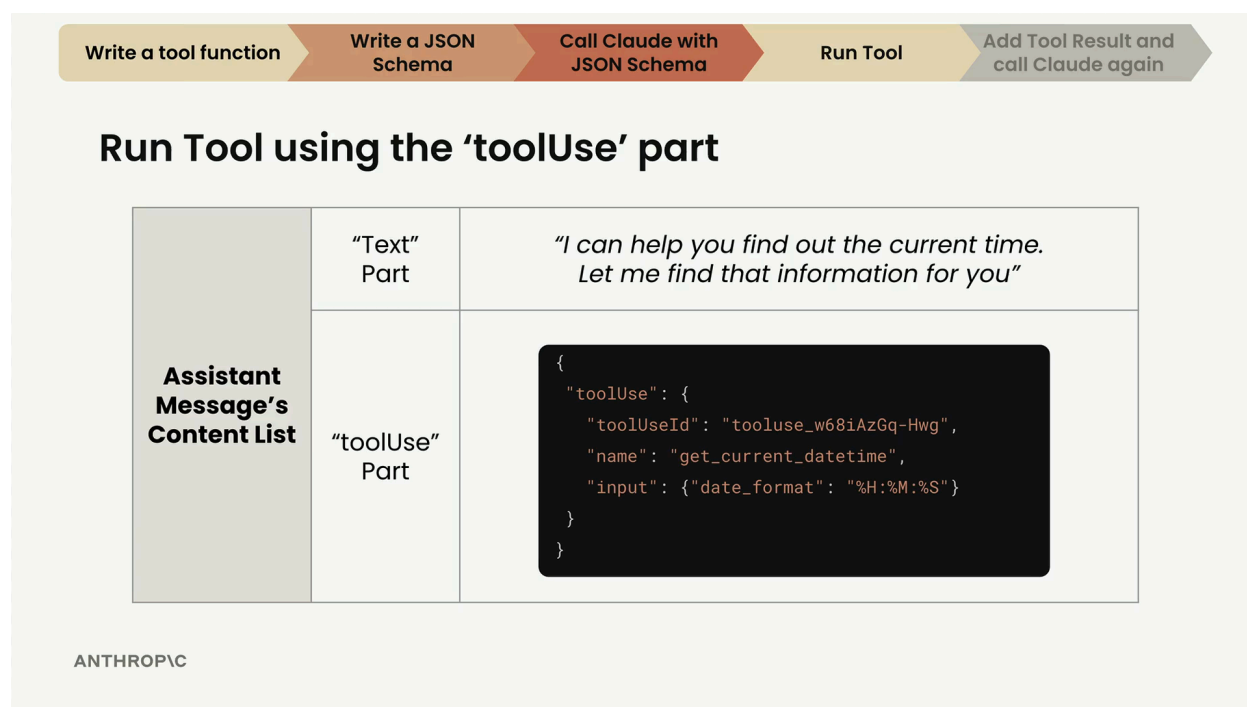
With these patterns in place, you're ready to handle Claude's tool use requests and maintain proper conversation flow throughout multi-turn tool interactions.

Running tool functions

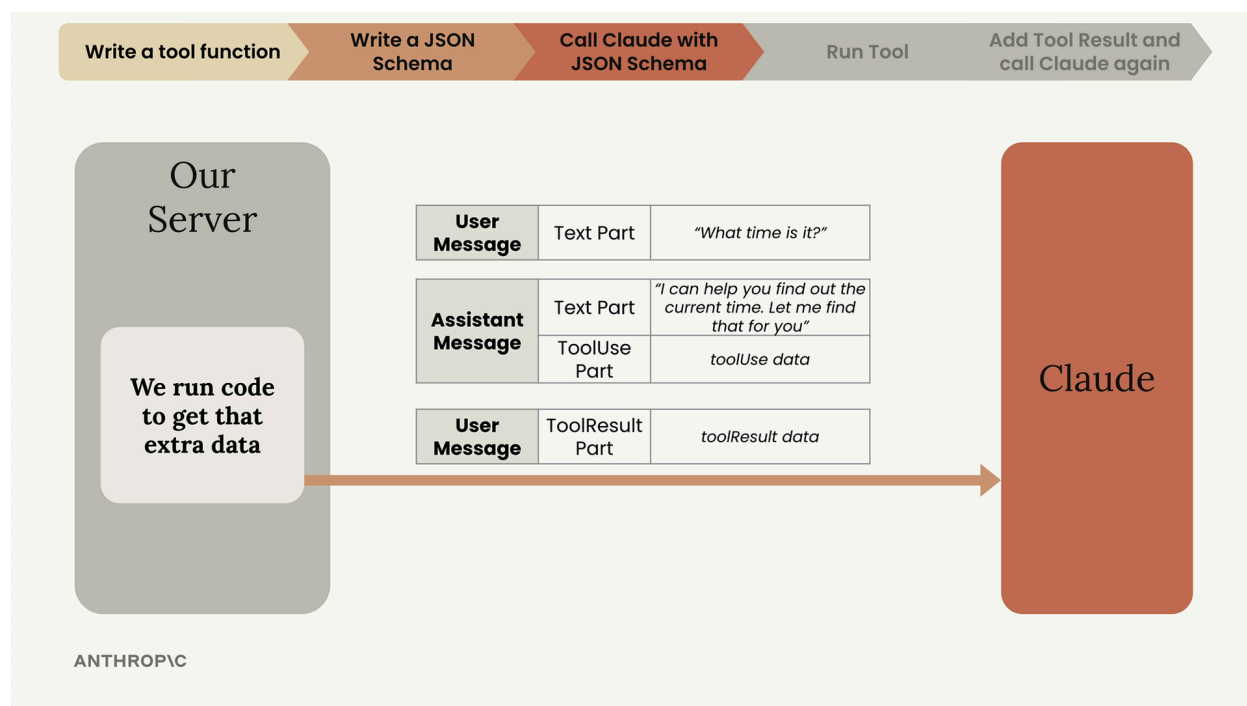
When Claude responds with a tool use request, your server needs to actually run the requested tool and send the results back. This step involves extracting tool use parts from Claude's response, executing the appropriate functions, and formatting the results properly.

Handling Multiple Tool Requests

Claude can send multiple tool use parts in a single response. Your code needs to handle this possibility defensively. An assistant message might contain a text part followed by one, two, or even more tool use parts.



The flow works like this: Claude sends a request with JSON schema, receives a tool use part, then your server runs the tool and sends back a tool result part for Claude to provide a final response.



Extracting Tool Use Parts

First, create a function to process all the parts returned from a chat request:

```
def run_tools(parts):
    tool_requests = [part for part in parts if "toolUse" in part]
    tool_result_parts = []

    for tool_request in tool_requests:
        tool_use_id = tool_request["toolUse"]["toolUseId"]
        tool_name = tool_request["toolUse"]["name"]
        tool_input = tool_request["toolUse"]["input"]
```

This comprehension filters the parts list to only include dictionaries that contain a "toolUse" key, ignoring text parts.

Running the Actual Tools

Create a helper function to execute the requested tool:

```
def run_tool(tool_name, tool_input):
    if tool_name == "get_current_datetime":
        return get_current_datetime(**tool_input)
    else:
        raise Exception(f"Unknown tool name: {tool_name}")
```

The key detail here is using `**tool_input` to splat the dictionary of arguments into your tool function. Claude always returns arguments as a dictionary object, so you need to unpack it properly.

Creating Tool Result Parts

After running a tool, you need to format the response as a tool result part:

Write a tool function

Write a JSON
Schema

Call Claude with
JSON Schema

Run Tool

Add Tool Result and
call Claude again

Note: There might be multiple 'toolUse' parts!

Assistant Message's Content List	"Text" Part	<i>"I can help you find out the current time. Let me find that information for you"</i>
	"toolUse" Part One	<pre>{ "toolUse": { "toolUseId": "tooluse_w68iAzGq-Hwg", "name": "get_current_datetime", "input": {"date_format": "%H:%M:%S"} } }</pre>
	"toolUse" Part Two	<pre>{ "toolUse": { "toolUseId": "tooluse_1iJPap3-MQP", "name": "other_tool", "input": {"value": "hello"} } }</pre>

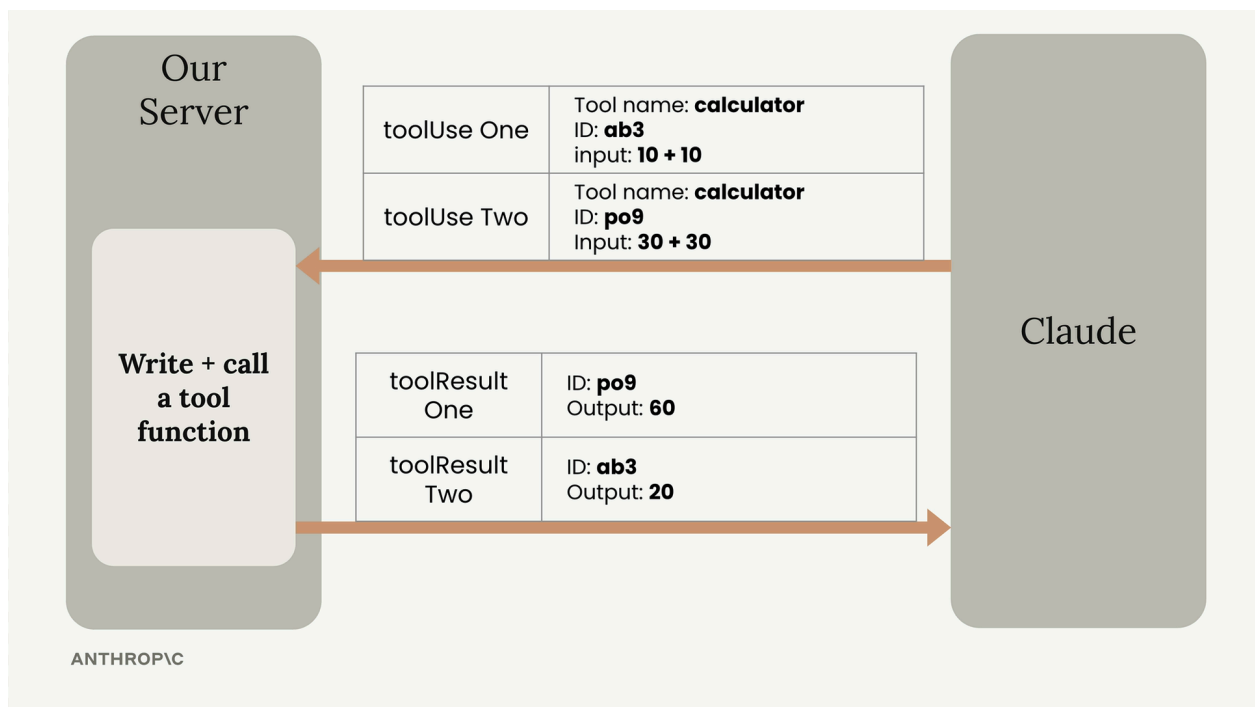
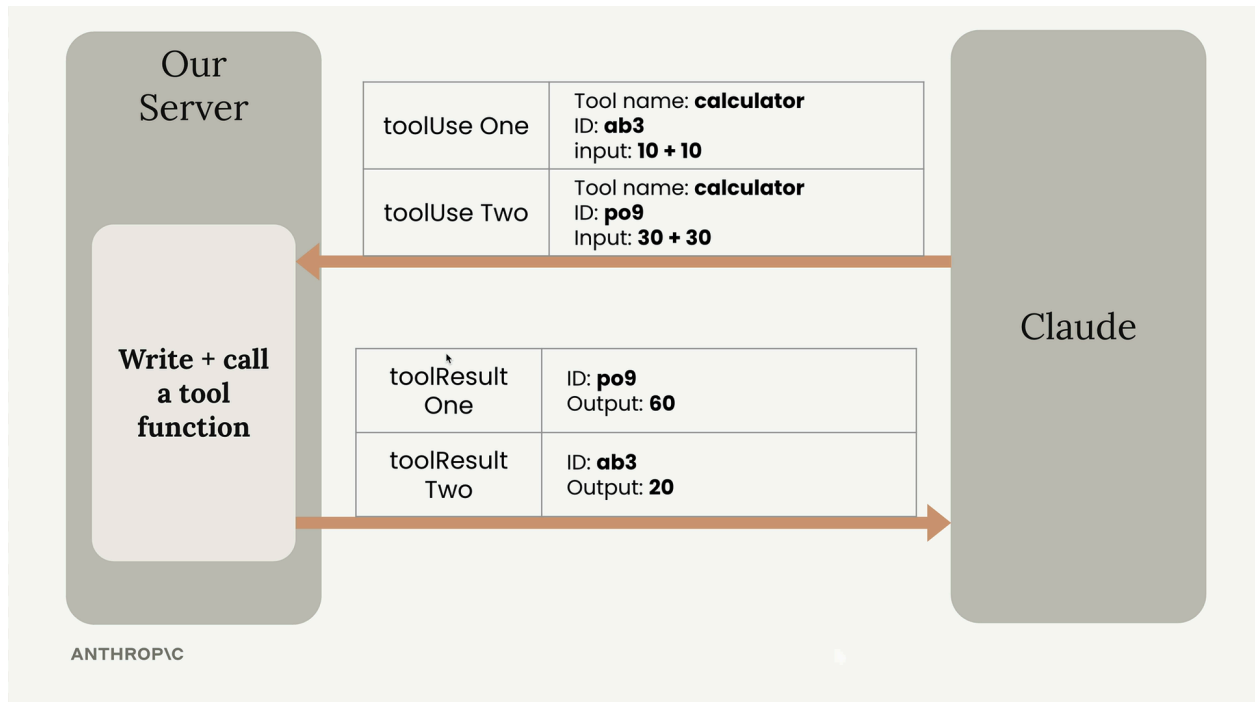
ANTHROPIC

Tool result parts require three key properties:

- **toolUseId** - Must match the original tool use part's ID
- **content** - The output from your tool, serialized as a string
- **status** - Either "success" or "error"

Understanding Tool Use IDs

The tool use ID system becomes important when Claude requests multiple tools in parallel. For example, if Claude wants to run a calculator tool twice:



Each tool use gets a unique ID (like "ab3" and "po9"), and your tool results must include the matching IDs so Claude knows which result corresponds to which request.

Error Handling

Wrap your tool execution in try-catch blocks. Claude is intelligent about tool errors and might adjust its approach if you return proper error information:

```
try:
    tool_output = run_tool(tool_name, tool_input)
    tool_result_part = {
        "toolResult": {
            "toolUseId": tool_use_id,
            "content": [{"text": json.dumps(tool_output)}],
            "status": "success"
        }
    }
except Exception as e:
    tool_result_part = {
        "toolResult": {
            "toolUseId": tool_use_id,
            "content": [{"text": f"Error: {e}"}],
            "status": "error"
        }
    }
```

Complete Implementation

Here's the full function that processes tool requests and returns formatted results:

```
def run_tools(parts):
    tool_requests = [part for part in parts if "toolUse" in part]
    tool_result_parts = []

    for tool_request in tool_requests:
        tool_use_id = tool_request["toolUse"]["toolUseId"]
        tool_name = tool_request["toolUse"]["name"]
        tool_input = tool_request["toolUse"]["input"]

        try:
            tool_output = run_tool(tool_name, tool_input)
            tool_result_part = {
                "toolResult": {
                    "toolUseId": tool_use_id,
                    "content": [{"text": json.dumps(tool_output)}],
                    "status": "success"
                }
            }
        except Exception as e:
```

```

tool_result_part = {
  "toolResult": {
    "toolUseId": tool_use_id,
    "content": [{"text": f"Error: {e}"}],
    "status": "error"
  }
}

```

```

tool_result_parts.append(tool_result_part)

```

```

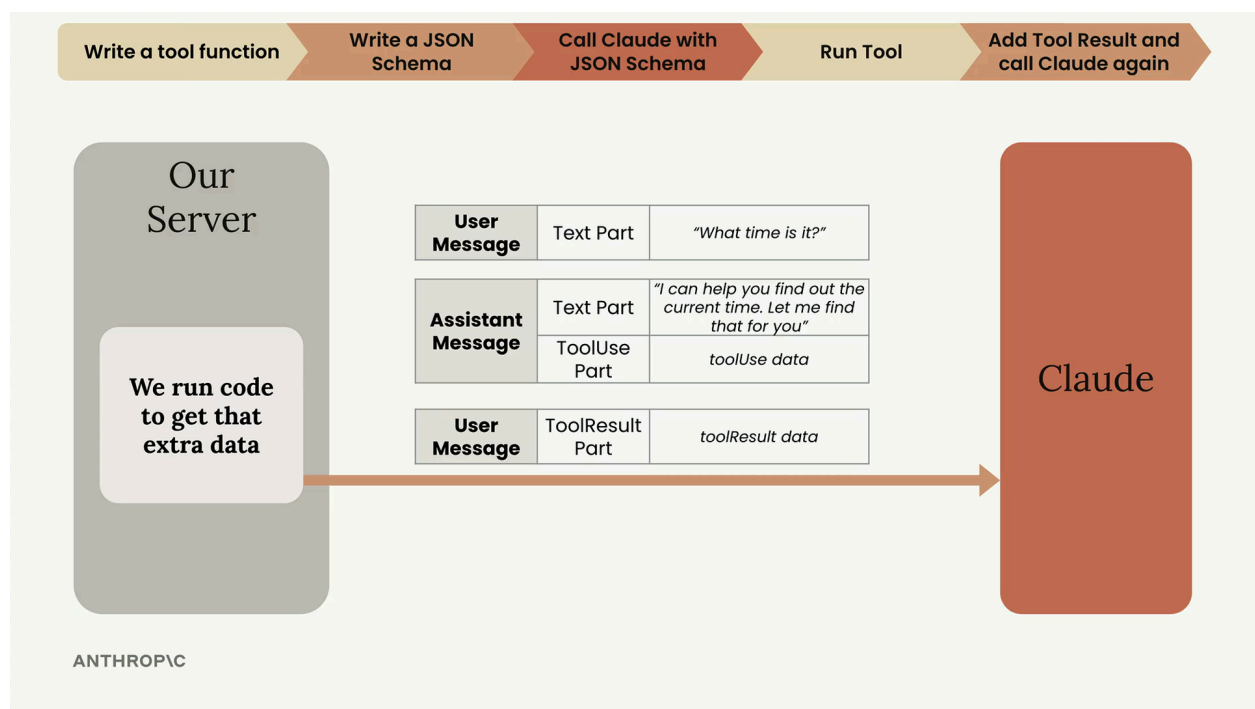
return tool_result_parts

```

Once you have the tool result parts, you can send them back to Claude in your next chat request, completing the tool use cycle.

Sending tool results

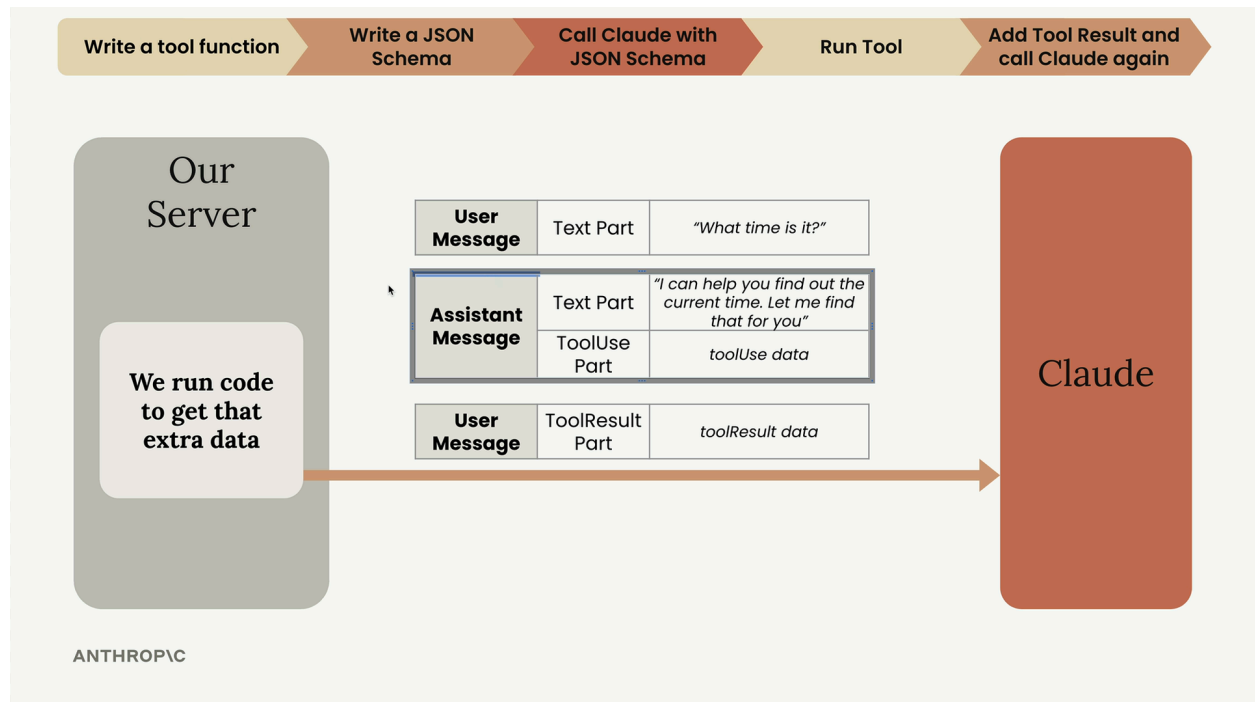
Now we're at the final step of the tool use workflow. After running our tools and getting the results, we need to send everything back to Claude so it can provide a complete response to the user.



The process is straightforward: take all the tool result parts we generated, package them into a user message, and send the entire conversation history back to Claude along with the original tool schemas.

Adding the Assistant Message

First, we need to make sure our conversation history is complete. After Claude's initial response with the tool use request, we need to add that response to our message history using `add_assistant_message()`.



This ensures we have the complete conversation flow: user question → assistant tool request → tool results → final assistant response.

Running Tools and Creating Tool Results

The `run_tools()` function processes all the tool use requests from Claude's response and creates properly formatted tool result parts. Each tool result includes:

- The tool use ID (matching the original request)
- The actual output from running the tool
- A status indicating success or error


```
001_tools.ipynb •
001_tools.ipynb
Code + Markdown | ▶ Run All | ⌂ Restart | 🗑 Clear All Outputs | 📄 Jupyter Variables | 📖 Outline | ... | .venv (Python 3.12)

tool_requests = [part for part in parts if "toolUse" in part]
tool_result_parts = []

for tool_request in tool_requests:
    tool_use_id = tool_request["toolUse"]["toolUseId"]
    tool_name = tool_request["toolUse"]["name"]
    tool_input = tool_request["toolUse"]["input"]

    try:
        tool_output = run_tool(tool_name, tool_input)
        tool_result_part = {
            "toolResult": {
                "toolUseId": tool_use_id,
                "content": [{"text": json.dumps(tool_output)}],
                "status": "success",
            }
        }
    except Exception as e:
        tool_result_part = {
            "toolResult": {
                "toolUseId": tool_use_id,
                "content": [{"text": f"Error: {e}"}],
                "status": "error",
            }
        }
```

The function handles both successful tool executions and errors gracefully, wrapping everything in the correct JSON structure that Claude expects.

Adding Tool Results to the Conversation

Once we have our tool results, we add them to the conversation using `add_user_message()`:

```
add_user_message(messages, run_tools(parts))
```

This creates a user message containing all the tool result parts. The conversation now has the complete back-and-forth needed for Claude to provide a final response.

```
001_tools.ipynb •
001_tools.ipynb
Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | .venv (Python 3.12)

    'name': 'get_current_datetime',
    'input': {'date_format': '%Y-%m-%d %H:%M:%S'}}}}]]

add_user_message(messages, run_tools(parts))
messages

0.0s Python

[{'role': 'user', 'content': [{'text': 'What time is it?'}]},
 {'role': 'assistant',
  'content': [{'text': 'I can check the current time for you. Let me retrieve that information.'},
   {'toolUse': {'toolUseId': 'tooluse_YgXKBs2PQKicYzzR3x-40g',
    'name': 'get_current_datetime',
    'input': {'date_format': '%Y-%m-%d %H:%M:%S'}}}}],
 {'role': 'user',
  'content': [{'toolResult': {'toolUseId': 'tooluse_YgXKBs2PQKicYzzR3x-40g',
   'content': [{'text': '"2025-04-03 12:54:00"'}],
   'status': 'success'}}]}]]
```

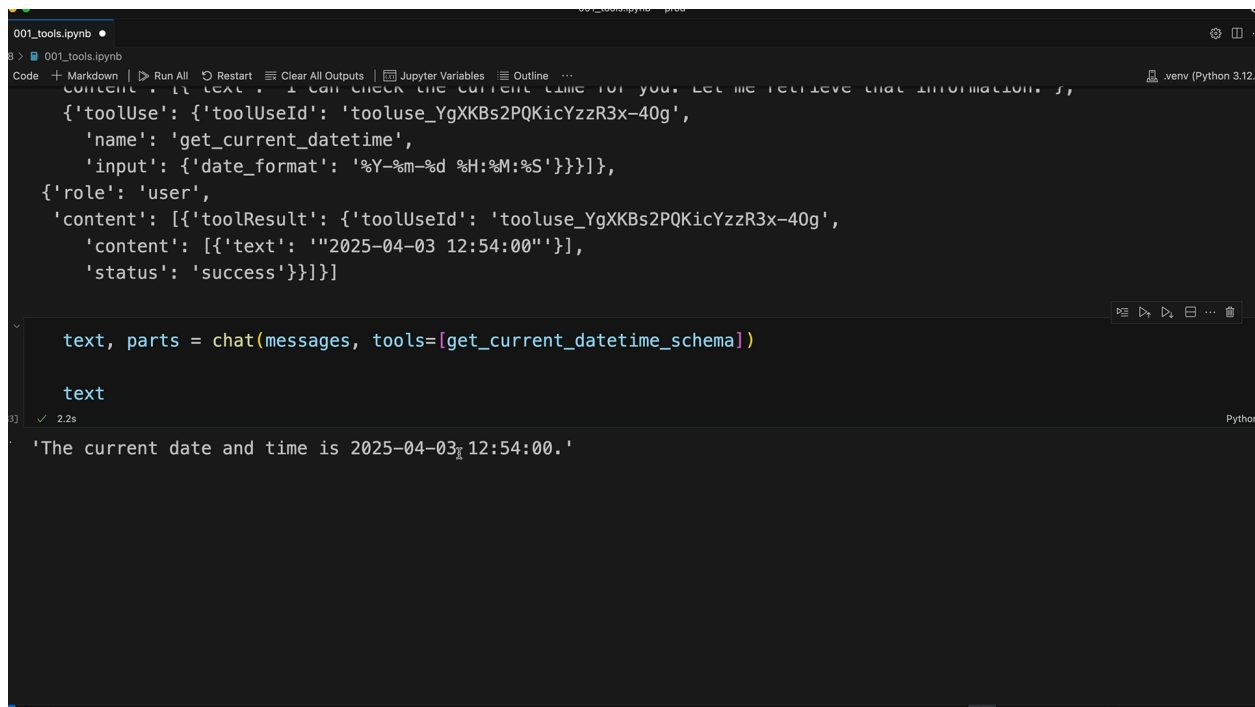
Final Call to Claude

The last step is sending everything back to Claude. This requires two important elements:

- The complete message history (user → assistant → user)
- The original tool schemas

```
text, parts = chat(messages, tools=[get_current_datetime_schema])
```

Including the tool schemas is crucial. Without them, Claude would be confused about the tool references in the conversation history and wouldn't understand what `get_current_datetime` actually does.



```
001_tools.ipynb •
001_tools.ipynb
Code + Markdown + Run All + Restart + Clear All Outputs + Jupyter Variables + Outline + ... .venv (Python 3.12)

content: [{"text": "I can check the current time for you. Let me retrieve that information.",
  'toolUse': {'toolUseId': 'tooluse_YgXKBs2PQKicYzzR3x-40g',
    'name': 'get_current_datetime',
    'input': {'date_format': '%Y-%m-%d %H:%M:%S'}}}],
{'role': 'user',
  'content': [{'toolResult': {'toolUseId': 'tooluse_YgXKBs2PQKicYzzR3x-40g',
    'content': [{'text': '"2025-04-03 12:54:00"'}],
    'status': 'success'}]}]}

text, parts = chat(messages, tools=[get_current_datetime_schema])

text

✓ 2.2s Python
'The current date and time is 2025-04-03 12:54:00.'
```

Success

When everything works correctly, Claude receives the tool results and can provide a complete, informed response. In our example, Claude successfully retrieved the current time and formatted it in a natural response: "The current date and time is 2025-04-03, 12:54:00."

This demonstrates that our tool integration is working properly. While Claude knows the current date, it doesn't have access to real-time information like the exact current time - which is exactly what our tool provided.

The complete tool use cycle is now working: Claude requests a tool, we execute it, return the results, and Claude incorporates that information into its final response to the user.

Multi Turn Conversations with tools

Building multi-turn conversations with tool use requires handling different response types from Claude. When Claude responds, it might need to use a tool, or it might provide a direct answer. Your code needs to handle both scenarios gracefully.

The Problem with Simple Tool Integration

If you just add tool results to every conversation, you'll run into issues. When Claude answers a simple question like "What is 1+1?", it doesn't need any tools. But if your code always tries to process tool results, you'll end up adding empty messages to your conversation history.

The solution is to check the `stop_reason` that comes back with every Claude response. This tells you why Claude stopped generating - whether it finished naturally or because it wants to use a tool.

Stop Reasons

Write a tool function

Write a JSON Schema

Call Claude with JSON Schema

Run Tool

Add Tool Result and call Claude again

Stop Reason

"tool_use"	The model has decided that it needs to call a tool
"end_turn"	Model has finished generating it's assistant message
"max_tokens"	The model has hit its output limit and can't generate any more output
"stop_sequence"	The model has encountered one of your provided stop sequences

ANTHROPIC

Claude can stop for several reasons:

- `"tool_use"` - The model wants to call a tool
- `"end_turn"` - Model finished generating its response
- `"max_tokens"` - Hit the output limit
- `"stop_sequence"` - Encountered a stop sequence you provided

Improving the Chat Function

First, update your chat function to return more information. Instead of just returning text and parts separately, return a dictionary with everything you need:

```
def chat(messages, tools=None, system=None, **kwargs):
    # ... existing code ...

    return {
        "parts": parts,
        "stop_reason": response["stopReason"],
```

```

    "text": "\n".join([p["text"] for p in parts if "text" in p])
}

```

This approach extracts all text content from the response parts, which is more robust than assuming the first part is always text.

Building a Conversation Loop

Create a function that handles the full conversation flow:

```

def run_conversation(messages):
    while True:
        result = chat(messages, tools=[get_current_datetime_schema])

        add_assistant_message(messages, result["parts"])
        print(result["text"])

        if result["stop_reason"] != "tool_use":
            break

        tool_result_parts = run_tools(result["parts"])
        add_user_message(messages, tool_result_parts)

    return messages

```

This loop continues until Claude stops for a reason other than tool use. Each iteration:

1. Sends the current messages to Claude
2. Adds Claude's response to the message history
3. Checks if Claude wants to use a tool
4. If so, runs the tools and adds results back to the conversation
5. If not, exits the loop

Testing the Implementation

This approach handles both tool-requiring and simple questions:

```

# Tool-requiring question
messages = []
add_user_message(messages, "What time is it?")
run_conversation(messages)

```

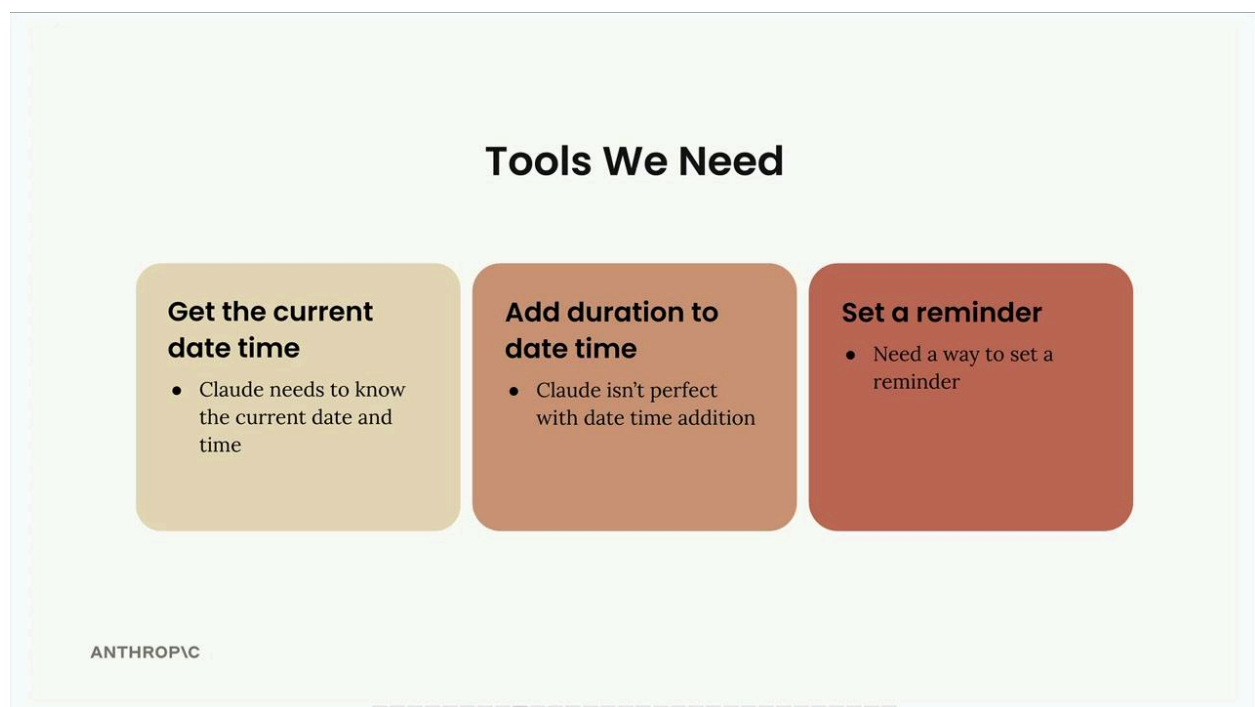
```
# Simple question
messages = []
add_user_message(messages, "What is 1+1?")
run_conversation(messages)
```

For time questions, Claude will use the datetime tool. For math questions, it responds directly without any tool calls. The conversation loop adapts automatically based on Claude's stop reason.

This pattern scales well when you add more tools - the same loop handles any combination of tool use and direct responses, making your conversational AI more robust and natural

Adding Multiple tools

Now that we have one tool working, it's time to add the remaining two tools to complete our project: `add_duration_to_datetime` and `set_reminder`. The good news is that once you have the foundation in place, adding new tools is straightforward.

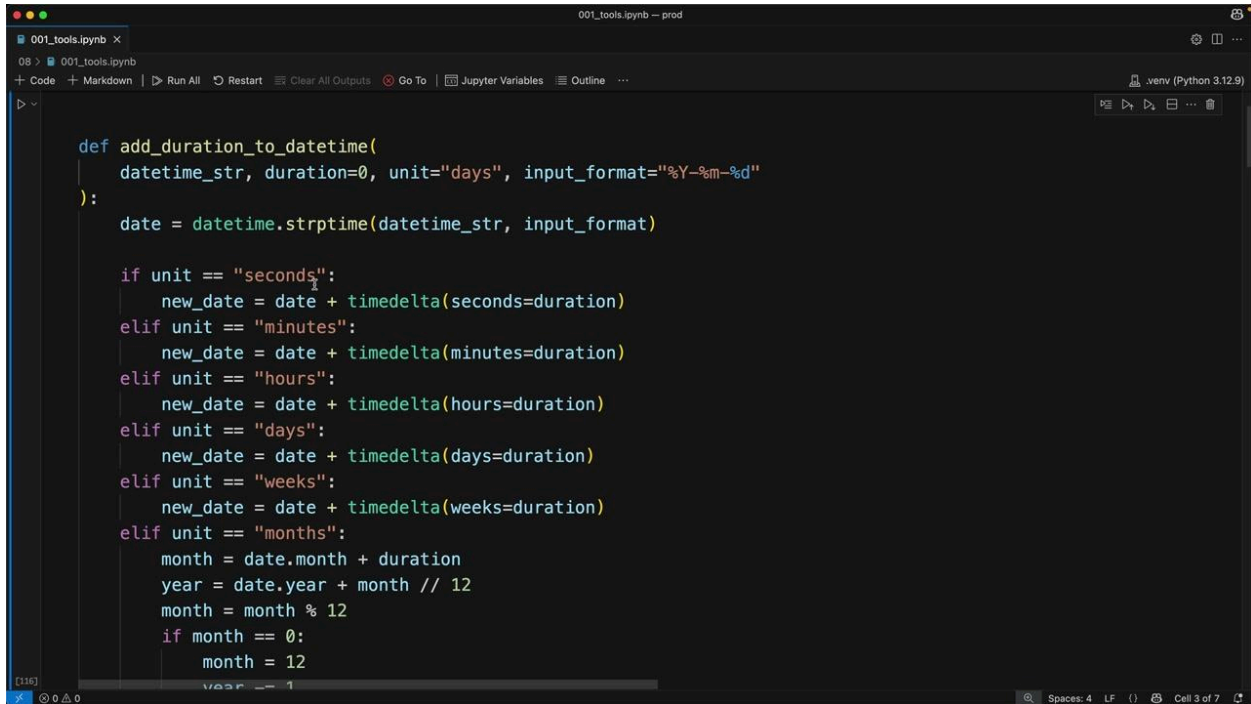


Pre-built Functions and Schemas

To save time, the implementations for both additional functions are already provided, along with their JSON schema specifications. You can find these in the earlier code cells:

- `add_duration_to_datetime` - Handles date arithmetic for various time units

- **set_reminder** - Creates reminders (currently just prints output, but could be extended to integrate with actual reminder systems)



```
def add_duration_to_datetime(
    datetime_str, duration=0, unit="days", input_format="%Y-%m-%d"
):
    date = datetime.strptime(datetime_str, input_format)

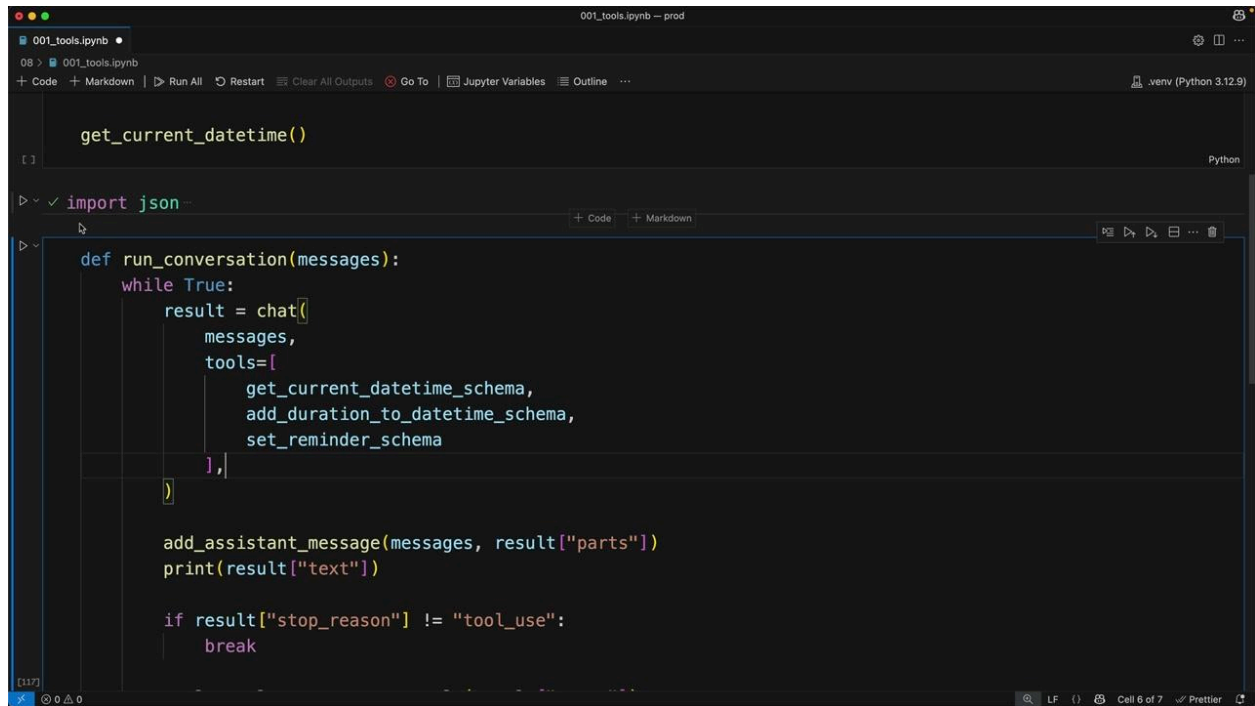
    if unit == "seconds":
        new_date = date + timedelta(seconds=duration)
    elif unit == "minutes":
        new_date = date + timedelta(minutes=duration)
    elif unit == "hours":
        new_date = date + timedelta(hours=duration)
    elif unit == "days":
        new_date = date + timedelta(days=duration)
    elif unit == "weeks":
        new_date = date + timedelta(weeks=duration)
    elif unit == "months":
        month = date.month + duration
        year = date.year + month // 12
        month = month % 12
        if month == 0:
            month = 12
        new_date = datetime(year=date.year + year, month=month, day=1)
```

Each function comes with a corresponding JSON schema that defines the expected parameters and their types.

Adding Tools to the Conversation

The first step is to include the new tool schemas in your conversation function. In the `run_conversation` function, add the additional schemas to the tools array:

```
tools=[
    get_current_datetime_schema,
    add_duration_to_datetime_schema,
    set_reminder_schema
]
```



```
get_current_datetime()

import json

def run_conversation(messages):
    while True:
        result = chat(
            messages,
            tools=[
                get_current_datetime_schema,
                add_duration_to_datetime_schema,
                set_reminder_schema
            ],
        )

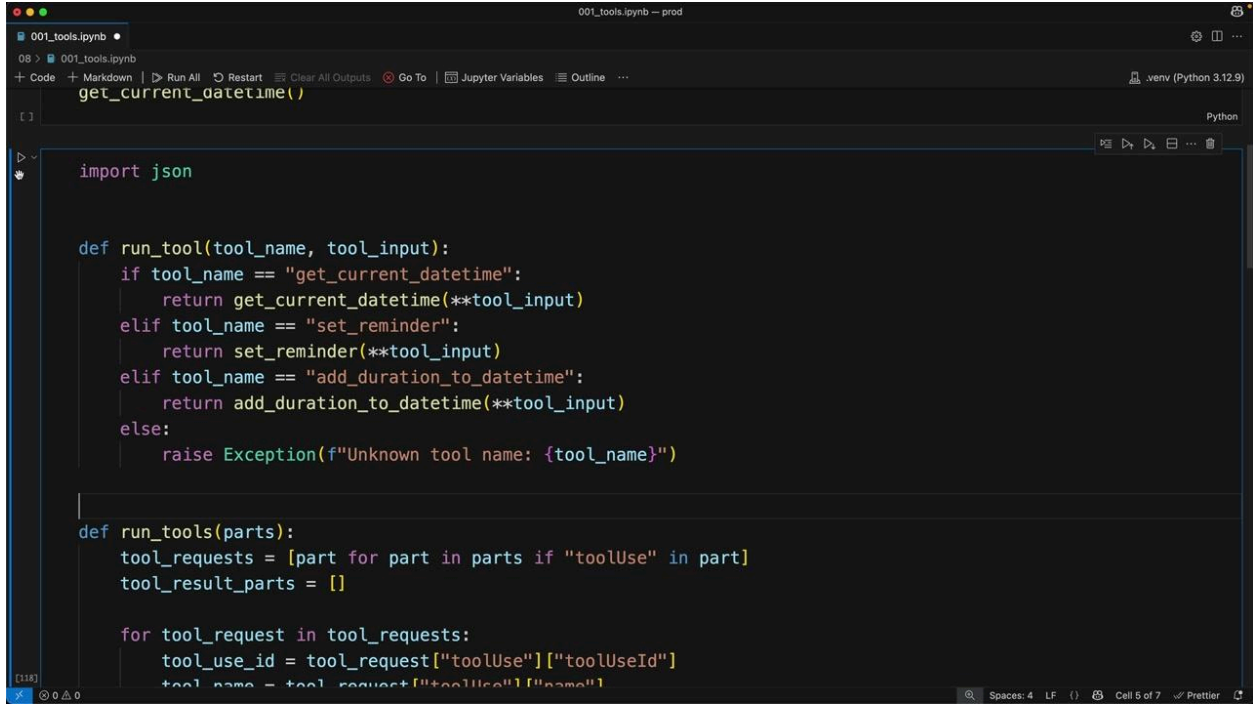
        add_assistant_message(messages, result["parts"])
        print(result["text"])

        if result["stop_reason"] != "tool_use":
            break
```

Wiring Up the Tool Functions

Next, you need to update the `run_tool` function to handle the new tool names. Add two additional conditional branches:

```
def run_tool(tool_name, tool_input):
    if tool_name == "get_current_datetime":
        return get_current_datetime(**tool_input)
    elif tool_name == "set_reminder":
        return set_reminder(**tool_input)
    elif tool_name == "add_duration_to_datetime":
        return add_duration_to_datetime(**tool_input)
    else:
        raise Exception(f"Unknown tool name: {tool_name}")
```

The screenshot shows a Jupyter Notebook window titled '001_tools.ipynb - prod'. The interface includes a top bar with navigation icons and a toolbar with options like 'Code', 'Markdown', 'Run All', 'Restart', 'Clear All Outputs', 'Go To', 'Jupyter Variables', and 'Outline'. The code is written in Python and defines two functions: `run_tool` and `run_tools`. The `run_tool` function handles specific tool requests like `get_current_datetime`, `set_reminder`, and `add_duration_to_datetime`. The `run_tools` function processes a list of tool requests and extracts the tool name and ID from each request.

```
get_current_datetime()

import json

def run_tool(tool_name, tool_input):
    if tool_name == "get_current_datetime":
        return get_current_datetime(**tool_input)
    elif tool_name == "set_reminder":
        return set_reminder(**tool_input)
    elif tool_name == "add_duration_to_datetime":
        return add_duration_to_datetime(**tool_input)
    else:
        raise Exception(f"Unknown tool name: {tool_name}")

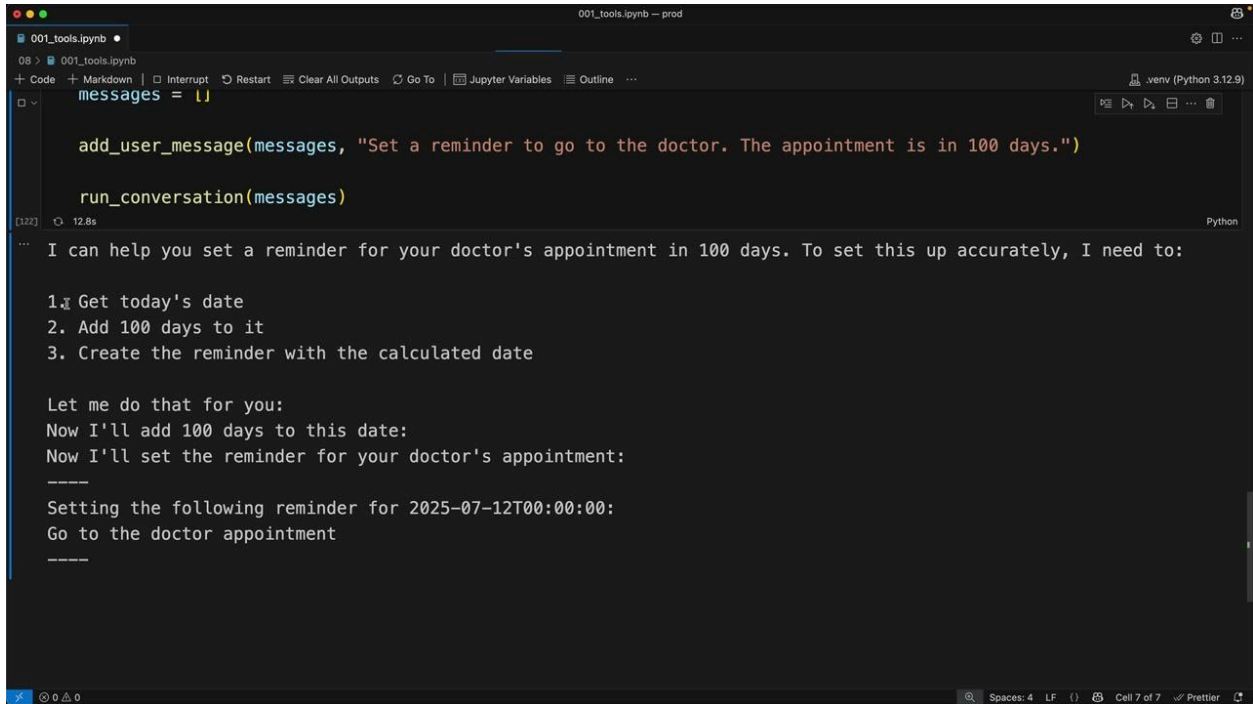
def run_tools(parts):
    tool_requests = [part for part in parts if "toolUse" in part]
    tool_result_parts = []

    for tool_request in tool_requests:
        tool_use_id = tool_request["toolUse"]["toolUseId"]
        tool_name = tool_request["toolUse"]["name"]
```

Testing the Complete System

With all tools connected, you can now test complex workflows that require multiple tool calls. For example, asking Claude to "Set a reminder to go to the doctor. The appointment is in 100 days" will trigger a sequence of operations:

1. Get today's date using `get_current_datetime`
2. Add 100 days to that date using `add_duration_to_datetime`
3. Create the reminder using `set_reminder`

A screenshot of a Jupyter Notebook interface. The top bar shows the file name '001_tools.ipynb' and the environment 'prod'. Below the top bar, there are tabs for 'Code', 'Markdown', 'Interrupt', 'Restart', 'Clear All Outputs', 'Go To', 'Jupyter Variables', and 'Outline'. The code cell contains two function calls: `add_user_message(messages, "Set a reminder to go to the doctor. The appointment is in 100 days.")` and `run_conversation(messages)`. The output cell shows a response from Claude, which includes a list of steps to set a reminder, a confirmation of the date, and a confirmation of the reminder being set. The output is as follows:

```
[122] 12.8s Python
... I can help you set a reminder for your doctor's appointment in 100 days. To set this up accurately, I need to:

1. Get today's date
2. Add 100 days to it
3. Create the reminder with the calculated date

Let me do that for you:
Now I'll add 100 days to this date:
Now I'll set the reminder for your doctor's appointment:
-----
Setting the following reminder for 2025-07-12T00:00:00:
Go to the doctor appointment
-----
```

Claude automatically breaks down the request into logical steps and explains its plan before executing each tool call. The output shows the complete workflow, including the calculated future date and confirmation of the reminder being set.

Key Takeaway

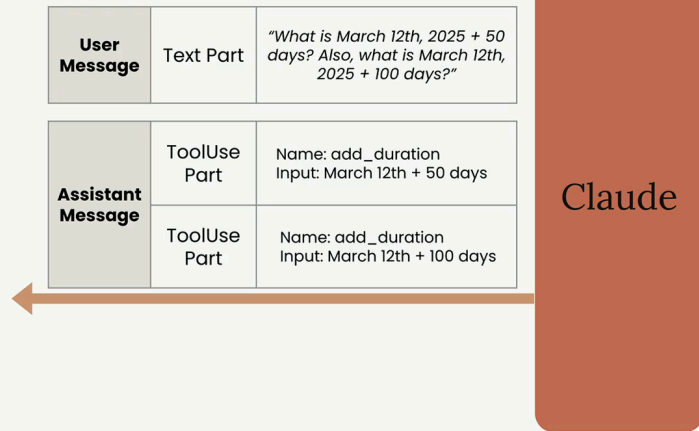
Once you have the foundational tool use infrastructure in place, adding new tools requires just two simple steps: including the schema in your tools array and adding a case to handle the tool name in your routing function. The initial setup might feel complex, but scaling to multiple tools becomes very manageable.

Batch Tool use

Claude can natively run multiple tools at the same time, but some versions don't take advantage of this as much as you might wish. You can greatly increase the chances of Claude making multiple tool calls in a single message by implementing a batch tool.

Batch Tool Use

- Claude *can* natively run multiple tools at the same time, **but some versions don't take advantage of this as much as you might wish**
- You can greatly increase the chances of Claude making multiple tool calls in a single message by implementing a **batch tool**



ANTHROPIC

When Claude sends back tool use parts in a message, there can be more than one tool request in a single response. For example, if you ask "What is March 12th, 2025 + 50 days? Also, what is March 12th, 2025 + 100 days?", Claude could theoretically send back two separate tool use parts - one for each calculation. These operations are completely parallelizable since they don't depend on each other.

However, Claude doesn't always try to parallelize tool calls as much as you'd expect. Instead of making both calls simultaneously, it often makes them sequentially, which is less efficient.

How the Batch Tool Works

The batch tool is implemented just like any other tool - you need a tool specification and a function to handle when it gets called. The key idea is to create a tool that can invoke multiple other tools simultaneously.

Here's the basic structure of the batch tool specification:

```
{
  "name": "batch_tool",
  "description": "Invoke multiple other tool calls simultaneously",
  "input_schema": {
    "type": "object",
    "properties": {
      "invocations": {
        "type": "array",
```

```

    "description": "The tool calls to invoke",
    "items": {
      "type": "object",
      "properties": {
        "name": {
          "type": "string",
          "description": "The name of the tool to invoke"
        },
        "arguments": {
          "type": "string",
          "description": "The arguments to the tool, encoded as a JSON string"
        }
      },
      "required": ["name", "arguments"]
    }
  },
  "required": ["invocations"]
}

```

The tool takes a list of invocations, where each invocation contains the name of a tool to call and its arguments (encoded as a JSON string).

Implementation

The batch tool implementation involves two main functions:

The `run_batch` Function

```

def run_batch(tool_input):
    batch_output = []
    for invocation in tool_input["invocations"]:
        tool_name = invocation["name"]
        args = json.loads(invocation["arguments"])

        tool_output = run_tool(tool_name, args)
        batch_output.append({"tool_name": tool_name, "output": tool_output})

    return batch_output

```

This function loops through each invocation, extracts the tool name and arguments, calls the appropriate tool using the existing `run_tool` function, and collects all the results.

Adding to run_tool

You also need to add a case to your main `run_tool` function:

```
elif tool_name == "batch_tool":  
    return run_batch(tool_input)
```

Note that unlike other tools, you pass `tool_input` directly without using the splat operator (`**`), since the batch tool needs to handle the raw input structure.

Results

When you implement the batch tool and run the same date calculation query, instead of seeing two separate tool calls in the message log, you'll see a single call to the batch tool. This single call contains both date calculations as sub-invocations, effectively parallelizing the operations.

The batch tool is particularly useful when you have multiple independent operations that can be executed simultaneously. By "tricking" Claude into using this pattern, you can significantly improve the efficiency of your tool-calling workflows.

Structured data with tools

Earlier in this course, we covered how to get structured output from Claude using message pre-fills and stop sequences. While that approach works well and is easy to set up, we can get more reliable output using tools. This method is more complex to implement, but it provides better consistency when extracting structured data like JSON.

Why Learn Both Approaches?

You might wonder why we didn't just start with tools if they're more reliable. The answer is simple: tools require significantly more setup and complexity. Having both techniques available gives you flexibility - sometimes you'll want the quick prompt-based approach, other times you'll need the reliability that tools provide.

How Tool-Based Structured Output Works

The core concept is straightforward: instead of asking Claude to format its response as JSON, you create a tool whose input parameters match the exact structure of data you want to extract. Claude then "calls" this tool with the extracted data as arguments.

Tools for Structured Data

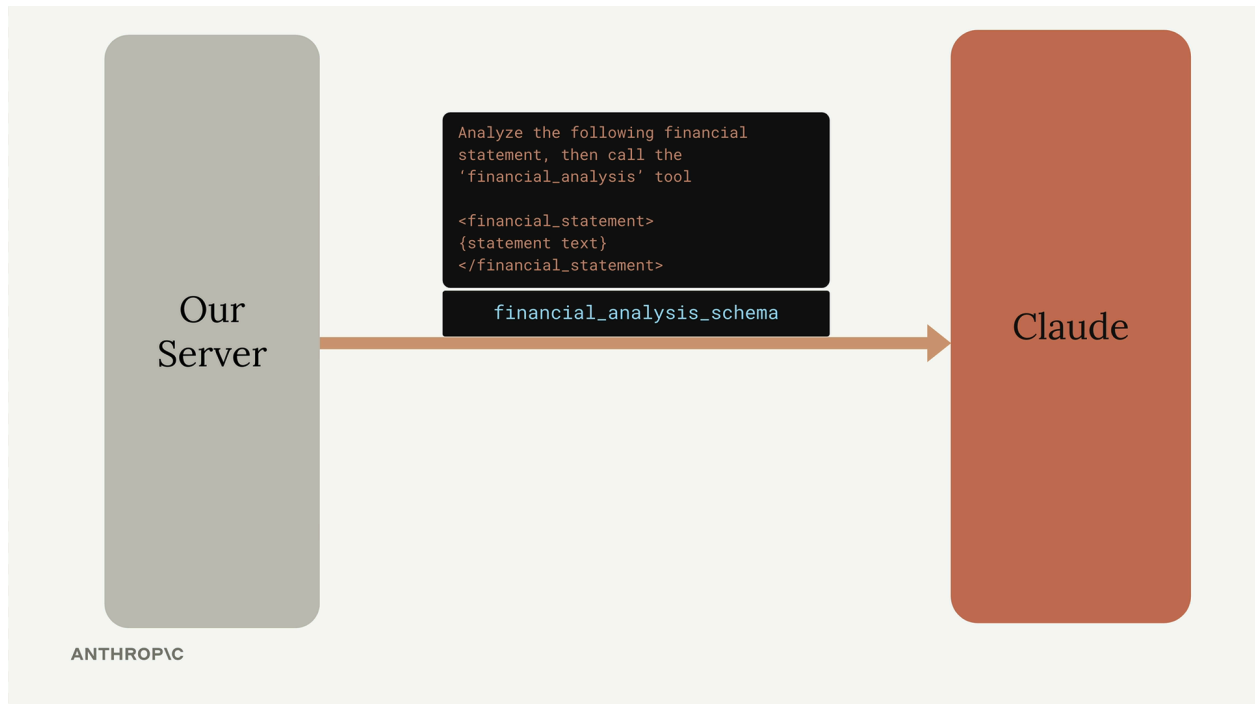
- Alternate method of extracting structured data
- More reliable, but far more complex
- Write a schema that describes the structure of data you're looking for
- Force Claude to use a tool with the 'toolChoice' param
- No need to provide a follow up!

ANTHROPIC

```
financial_analysis_schema = {
  "toolSpec": {
    "name": "analyze_financial_statement",
    "description": "Analyzes a financial statement and extracts key insights and metrics from the provided data.",
    "inputSchema": {
      "json": {
        "type": "object",
        "properties": {
          "balance": {
            "type": "integer",
            "description": "The current financial balance extracted from the statement."
          },
          "key_insights": {
            "type": "array",
            "items": {
              "type": "string"
            },
            "description": "List of key insights extracted from the financial statement."
          }
        },
        "required": ["balance", "key_insights"]
      }
    }
  }
}
```

Here's the process:

1. Write a JSON schema that describes the structure of data you want
2. Create a tool with that schema as its input specification
3. Send your data and the tool schema to Claude
4. Force Claude to use the tool with the `toolChoice` parameter
5. Extract the structured data from the tool call arguments



The flow looks like this: your server sends a prompt asking Claude to analyze data and call a specific tool. Claude responds with a tool use message containing the extracted JSON data. At that point, you simply take the data and end the conversation - no follow-up needed.

Controlling Tool Usage

When using tools for structured output, you want to guarantee that Claude uses your extraction tool. The `toolChoice` parameter gives you three options:

```
003_structured_data.ipynb •
003_structured_data.ipynb
Code + Markdown | ▶ Run All | ⏹ Restart | 🗑 Clear All Outputs | 📄 Jupyter Variables | 📖 Outline | ⚙️ .venv (Python 3.12)

messages,
    "Write a one-paragraph scholarly article about computer science. Include a title and author name",
)

result = chat(messages)

add_assistant_message(messages, result["text"])

result["text"]

Python

'# The Evolution of Computing Paradigms: From Von Neumann Architecture to Quantum Computing\n\nBy Dr. Eleanor J. Thornton\n\nThe field of computer science has witnessed exponential growth since the conceptualization of the Von Neumann architecture in the mid-20th century, evolving through distinct paradigm shifts that have fundamentally transformed computational capabilities and applications. This progression encompasses the transition from centralized mainframe computing to distributed systems, the advent of object-oriented programming methodologies, the emergence of parallel processing frameworks, and the recent developments in machine learning algorithms that enable systems to improve through experience rather than explicit programming. Contemporary research frontiers extend into quantum computing, where quantum bits leverage superposition and entanglement to potentially solve previously intractable problems, and neuromorphic computing architectures that emulate biological neural networks to achieve unprecedented energy efficiency. These advancements collectively suggest that the discipline continues to redefine itself through interdisciplinary collaboration, with implications extending far beyond traditional computational domains into fields such as biology, economics, linguistics, and cognitive science, fundamentally reshaping our understanding of information processing and problem-solving methodologies.'
```

- `{"toolChoice": {"auto": {}}}` - Model decides if it needs to use a tool (default)
- `{"toolChoice": {"any": {}}}` - Model must use a tool, but can choose which one
- `{"toolChoice": {"tool": {"name": "tool-name"}}}` - Model must use the specified tool

For structured output, you'll almost always want the third option to ensure Claude uses your extraction tool.

Practical Example

Let's say you want to extract the title, author, and key topics from an article. First, you'd create a tool schema:

```
article_details_schema = {
    "toolSpec": {
        "name": "article_details",
        "description": "Extracts key information from an article",
        "inputSchema": {
            "json": {
                "type": "object",
                "properties": {
                    "title": {
                        "type": "string",
                        "description": "The title of the article"
                    }
                }
            }
        }
    }
}
```



```

    },
    "author": {
      "type": "string",
      "description": "The author's name"
    },
    "topics": {
      "type": "array",
      "items": {"type": "string"},
      "description": "List of key topics mentioned"
    }
  },
  "required": ["title", "author", "topics"]
}
}
}
}

```

Then you'd call Claude with your data and force it to use the tool:

```

messages = []
add_user_message(messages, f"""
Analyze the article below and extract key data. Then call the article_details tool.

```

```

<article_text>
{article_text}
</article_text>
""")

```

```

result = chat(messages, tools=[article_details_schema], tool_choice="article_details")

```

Claude will respond with a tool use message containing the extracted data in the exact format you specified. The tool call arguments will contain your structured JSON data, ready to use in your application.

Key Benefits

- More reliable than prompt-based extraction
- Guaranteed structure matching your schema
- No need for message pre-fills or stop sequences
- Built-in validation through the tool schema

The main tradeoff is complexity - you need to write detailed schemas and handle tool responses. But when you need consistent, reliable structured output, tools are the way to go.

Flexible tool extraction

Writing detailed JSON schemas for structured data extraction can be a real pain point when working with AI tools. There's a clever workaround that lets you specify your desired data structure directly in your prompt instead of creating complex schemas.

The Flexible Schema Approach

Instead of writing a detailed schema for every data extraction task, you can create one generic tool called `to_json` that accepts any object structure. The key is setting the input schema to allow additional properties, then specifying your exact requirements in the prompt itself.

A Flexible Schema

- Allowed you to specify your desired JSON structure in the prompt, rather than the tool
- Removes a big pain point - writing a schema!
- Results won't be quite as good as writing a dedicated schema for your particular data structure

```
{
  "toolSpec": {
    "name": "to_json",
    "description": "This tool processes any JSON data and can be used for generating structured content, transforming information, or creating any JSON-based output needed for your task.",
    "inputSchema": {
      "json": {"type": "object", "additionalProperties": true}
    },
  },
}
```

ANTHROPIC

This approach removes a major pain point - constantly writing and managing large JSON schemas. The results won't be quite as good as a dedicated schema, but you'll still get high-quality JSON output with much less setup work.

How It Works

The process is straightforward:

- Create a single flexible schema that accepts any object structure
- In your prompt, specify exactly what data structure you want
- Tell Claude to call the `to_json` tool with your specified structure
- Use `tool_choice` to force Claude to use your tool

Setting Up the Prompt

When writing your prompt, be very explicit about the structure you want. Here's an example of how to structure your request:

Analyze the article below and extract key data. Then call the `to_json` tool.

```
<article_text>
{result["text"]}
</article_text>
```

When you call `to_json`, pass in the following structure:

```
{{
  "title": str # title of the article,
  "author": str # author of the article,
  "topics": List[str] # List of topics mentioned in the article
}}
```

Making the API Call

The API call uses the flexible schema and forces tool usage:

```
flexible_result = chat(messages, tools=[to_json_schema], tool_choice="to_json")
```

Easy Structure Changes

The real advantage becomes clear when you need to modify your data structure. Instead of rewriting an entire schema, you simply update your prompt. Want to add a field for the number of topics? Just add one line:

```
"num_topics": int # Number of topics mentioned
```

That's it - no schema modifications needed.

When to Use Each Approach

The flexible schema approach works great for:

- Rapid prototyping and experimentation
- Simple data extraction tasks

- Situations where you frequently change data requirements

Stick with dedicated schemas for:

- Critical production data extraction tasks
- Complex nested data structures
- When you need the highest possible accuracy

The flexible approach gives you about 90% of the quality with 10% of the setup work, making it perfect for most use cases where you need structured data extraction without the schema management overhead.

Text editor tool

Important Note: Up-to-date tool ID's for the text editor tool can be found in the AWS documentation here:

<https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-anthropic-claude-messages-tool-use.html#model-parameters-anthropic-anthropic-defined-tools>

The Text Editor Tool is Claude's built-in capability that gives it file system access and text editing abilities. Unlike other tools where you write both the schema and implementation, Claude already knows how to request text editor operations - you just need to handle those requests.

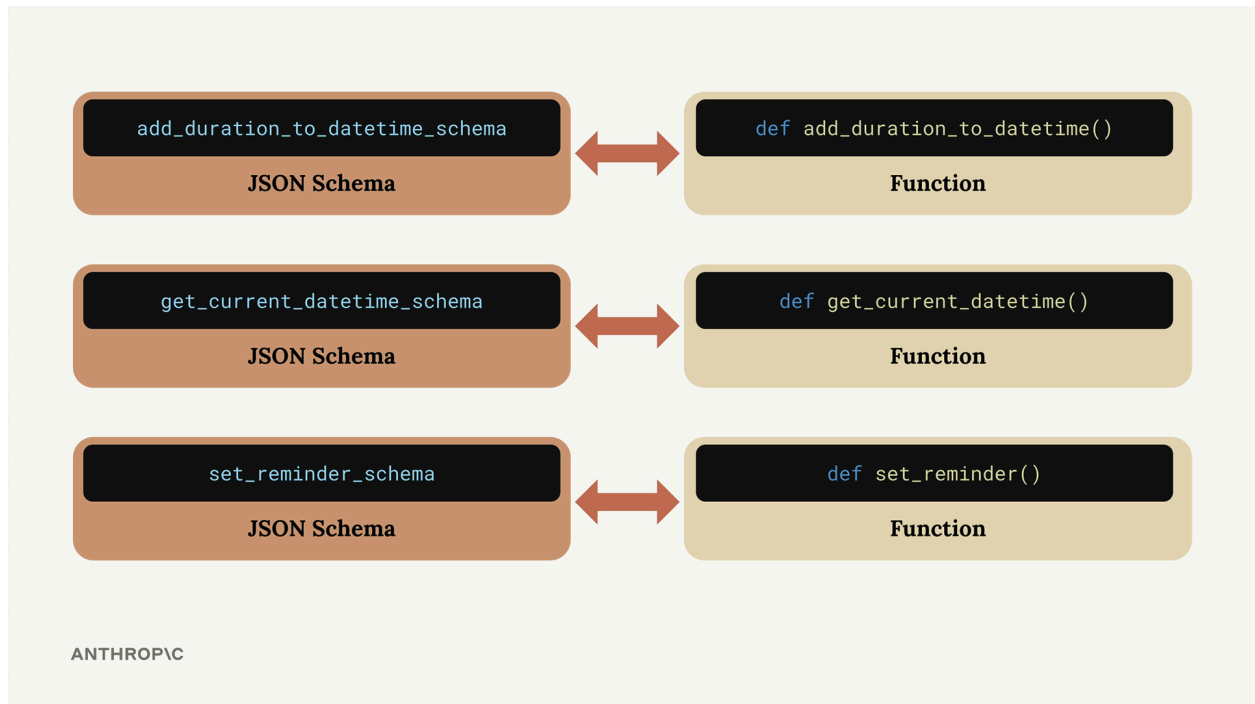
What the Text Editor Tool Does

This tool gives Claude the ability to work with files and directories like a software engineer would:

- View file or directory contents
- View specific ranges of lines in a file
- Replace text in files
- Create new files
- Insert text at specific line numbers
- Undo recent edits

How It Works

The Text Editor Tool is different from custom tools because only the JSON schema is built into Claude. You still need to provide the actual implementation.



When you create custom tools, you write both sides - the schema that tells Claude about the tool, and the function that actually does the work. With the Text Editor Tool, Claude already has the schema, but you must write functions to handle Claude's requests to view, edit, or create files.

Setting Up the Tool

To use the Text Editor Tool, you need to provide specific tool names that vary by Claude version:

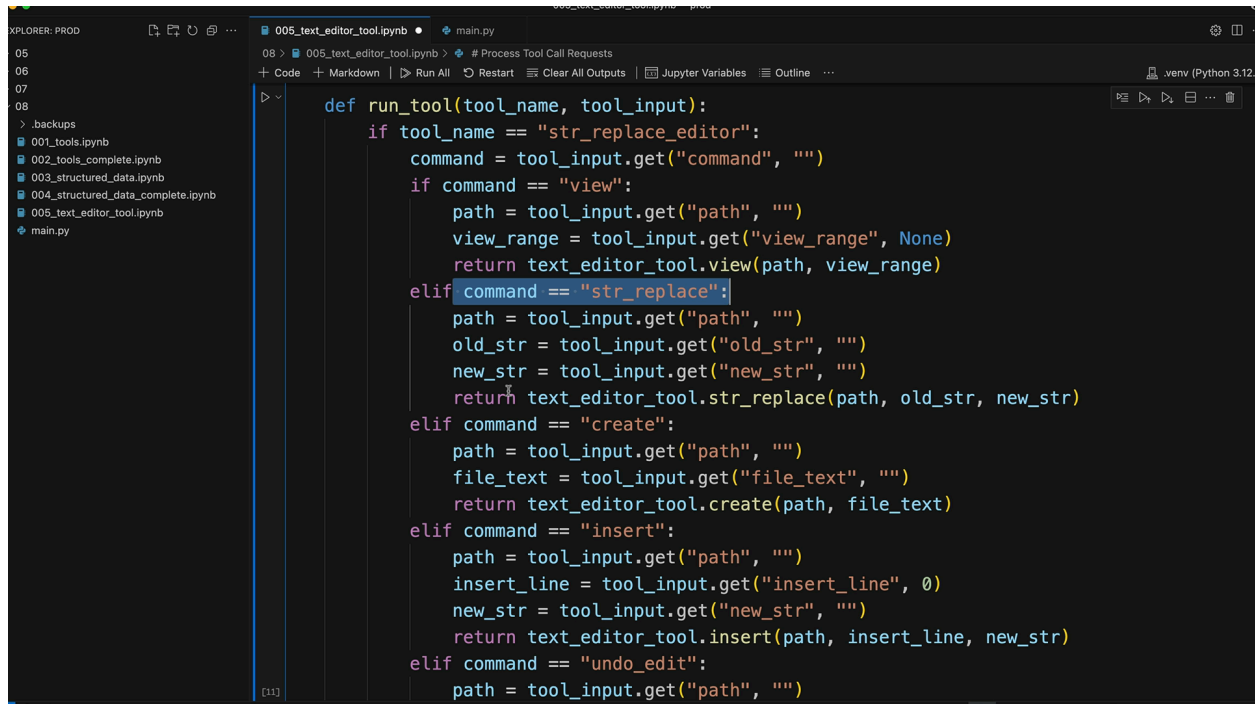
```
# For Claude 3.7
text_editor = "text_editor_20250124"
```

```
# For Claude 3.5
text_editor = "text_editor_20241022"
```

You'll also need to modify your chat function to accept the text editor parameter and include it in the model configuration.

Tool Commands

When Claude wants to use the text editor, it sends back tool use requests with specific commands:



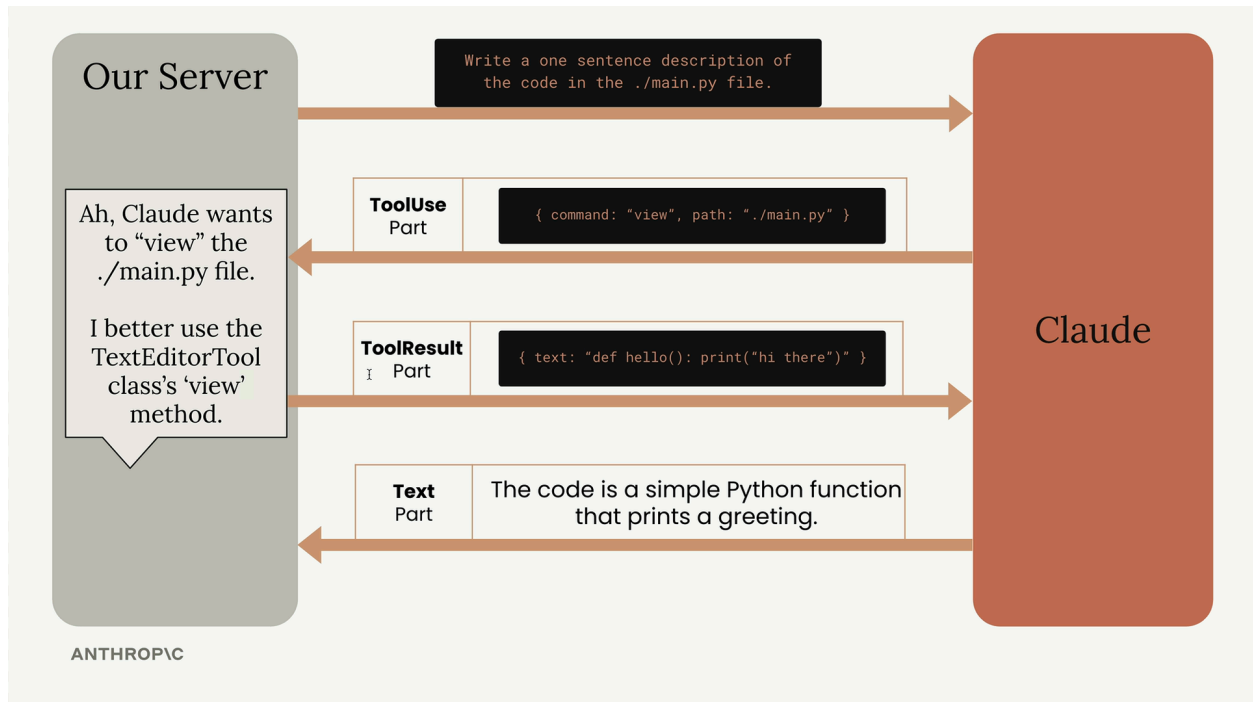
```
def run_tool(tool_name, tool_input):
    if tool_name == "str_replace_editor":
        command = tool_input.get("command", "")
        if command == "view":
            path = tool_input.get("path", "")
            view_range = tool_input.get("view_range", None)
            return text_editor_tool.view(path, view_range)
        elif command == "str_replace":
            path = tool_input.get("path", "")
            old_str = tool_input.get("old_str", "")
            new_str = tool_input.get("new_str", "")
            return text_editor_tool.str_replace(path, old_str, new_str)
        elif command == "create":
            path = tool_input.get("path", "")
            file_text = tool_input.get("file_text", "")
            return text_editor_tool.create(path, file_text)
        elif command == "insert":
            path = tool_input.get("path", "")
            insert_line = tool_input.get("insert_line", 0)
            new_str = tool_input.get("new_str", "")
            return text_editor_tool.insert(path, insert_line, new_str)
        elif command == "undo_edit":
            path = tool_input.get("path", "")
```

Your implementation needs to handle all five commands. Here's the basic structure for processing these requests:

```
def run_tool(tool_name, tool_input):
    if tool_name == "str_replace_editor":
        command = tool_input.get("command", "")
        if command == "view":
            path = tool_input.get("path", "")
            return text_editor_tool.view(path)
        elif command == "str_replace":
            path = tool_input.get("path", "")
            old_str = tool_input.get("old_str", "")
            new_str = tool_input.get("new_str", "")
            return text_editor_tool.str_replace(path, old_str, new_str)
        # ... handle other commands
```

Example: File Analysis

Here's how the tool works in practice. When you ask Claude to "Write a one sentence description of the code in the ./main.py file", this happens:



Claude sends a tool use request with `{"command": "view", "path": "./main.py"}`. Your server uses the `TextEditorTool` class to read the file and returns the contents. Claude then provides its analysis based on the code it read.

Practical Applications

The Text Editor Tool essentially turns Claude into a code assistant that can:

- Read existing code and provide analysis
- Create new files and functions
- Modify existing code
- Set up test files
- Refactor code across multiple files

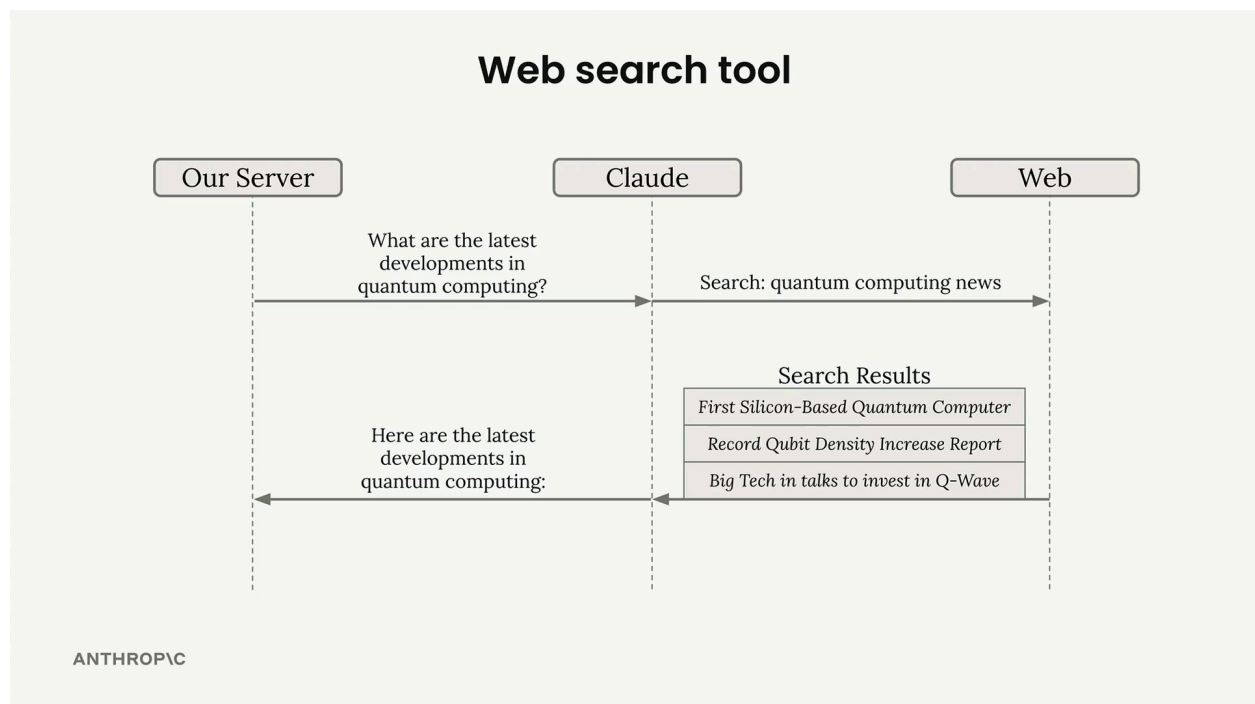
For example, you could ask Claude to "write a function to calculate pi to the 5th digit in main.py, then create a test.py file to test it." Claude will read the existing file, add the new function, create the test file, and write comprehensive tests - all automatically using the text editor commands.

This makes it possible to build AI-powered development tools similar to modern code editors with integrated AI features, where you can ask for code changes and have them implemented directly in your file system.

Web search tool

Important note: Your organization must enable the Web Search tool in the settings console before using it. You can find this setting here: <https://console.anthropic.com/settings/privacy>

Claude includes a built-in web search tool that lets it search the internet for current or specialized information to answer user questions. Unlike other tools where you need to provide the implementation, Claude handles the entire search process automatically - you just need to provide a simple schema to enable it.



Setting Up the Web Search Tool

To use the web search tool, you create a schema object with these required fields:

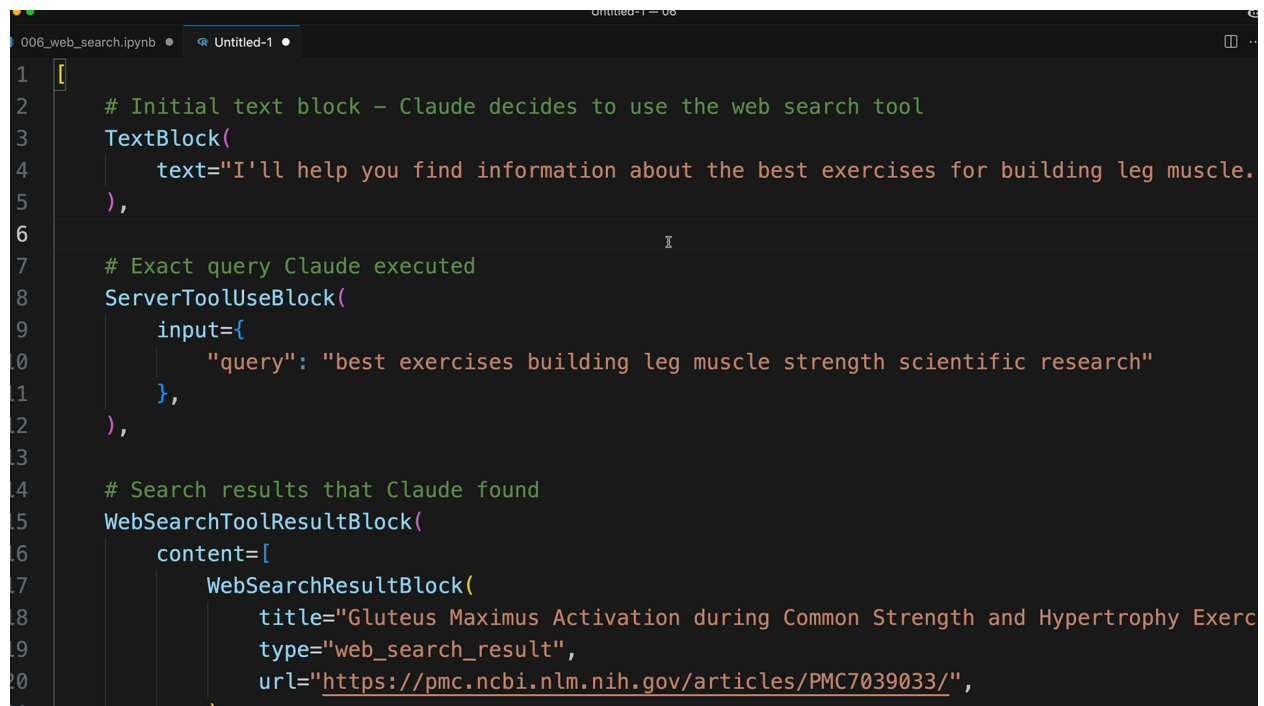
```
web_search_schema = {  
  "type": "web_search_20250305",  
  "name": "web_search",  
  "max_uses": 5  
}
```

The `max_uses` field limits how many searches Claude can perform. Claude might do follow-up searches based on initial results, so this prevents excessive API calls. A single search returns multiple results, but Claude may decide additional searches are needed.

How the Response Works

When Claude uses the web search tool, the response contains several types of blocks:

- **Text blocks** - Claude's explanation of what it's doing
- **ServerToolUseBlock** - Shows the exact search query Claude used
- **WebSearchToolResultBlock** - Contains the search results
- **WebSearchResultBlock** - Individual search results with titles and URLs
- **Citation blocks** - Text that supports Claude's statements

A screenshot of a Jupyter Notebook interface. The top bar shows two tabs: '006_web_search.ipynb' and 'Untitled-1'. The notebook content is a Python dictionary representing a response structure. It starts with a 'TextBlock' containing the text 'I'll help you find information about the best exercises for building leg muscle.'. This is followed by a 'ServerToolUseBlock' with an 'input' dictionary containing a 'query' about 'best exercises building leg muscle strength scientific research'. Finally, there is a 'WebSearchToolResultBlock' containing a list of 'WebSearchResultBlock' objects. One result is shown with a title 'Gluteus Maximus Activation during Common Strength and Hypertrophy Exerc...', a type of 'web_search_result', and a URL from 'https://pmc.ncbi.nlm.nih.gov/articles/PMC7039033/'.

```
1 {
2     # Initial text block - Claude decides to use the web search tool
3     TextBlock(
4         text="I'll help you find information about the best exercises for building leg muscle.",
5     ),
6
7     # Exact query Claude executed
8     ServerToolUseBlock(
9         input={
10             "query": "best exercises building leg muscle strength scientific research"
11         },
12     ),
13
14     # Search results that Claude found
15     WebSearchToolResultBlock(
16         content=[
17             WebSearchResultBlock(
18                 title="Gluteus Maximus Activation during Common Strength and Hypertrophy Exerc",
19                 type="web_search_result",
20                 url="https://pmc.ncbi.nlm.nih.gov/articles/PMC7039033/",
21             ),
22         ],
23     ),
24 }
```

The response structure lets you see exactly what Claude searched for and which sources it found. Citations include the specific text Claude used to support its answers, along with the source URLs.

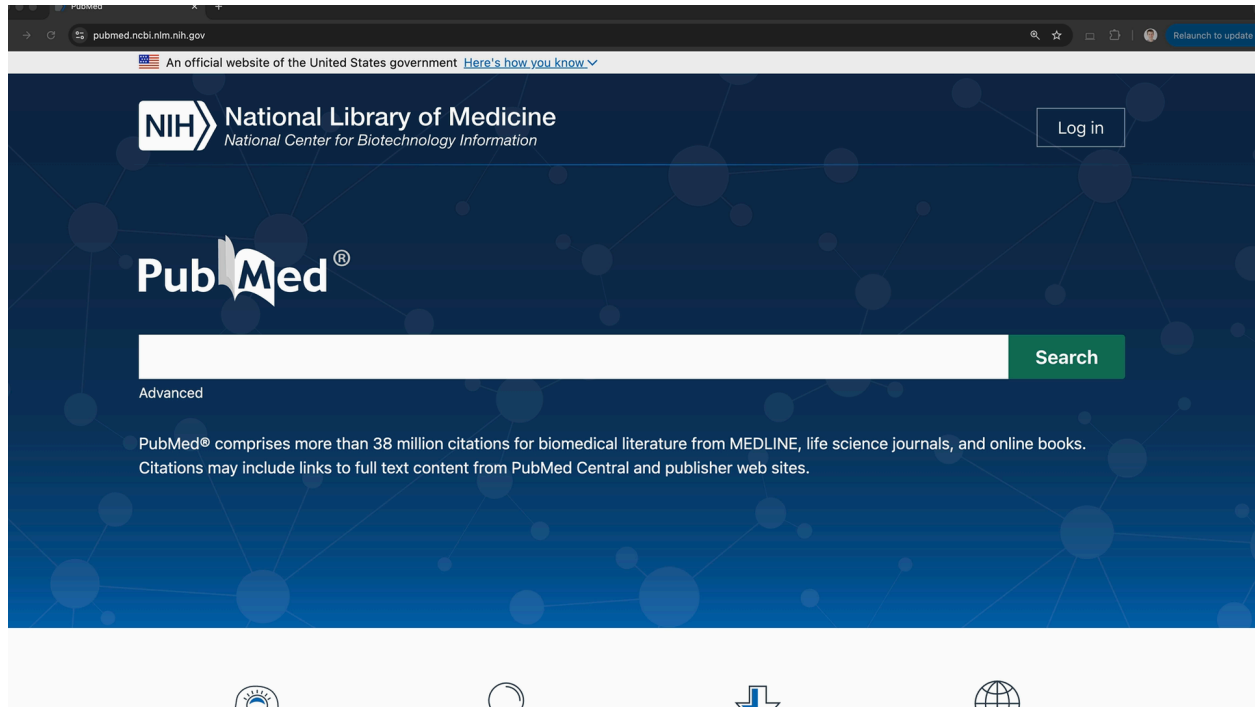
Restricting Search Domains

You can limit searches to specific domains using the `allowed_domains` field. This is particularly useful when you want reliable, authoritative sources:

```
web_search_schema = {
    "type": "web_search_20250305",
    "name": "web_search",
    "max_uses": 5,
    "allowed_domains": ["nih.gov"]
}
```

}

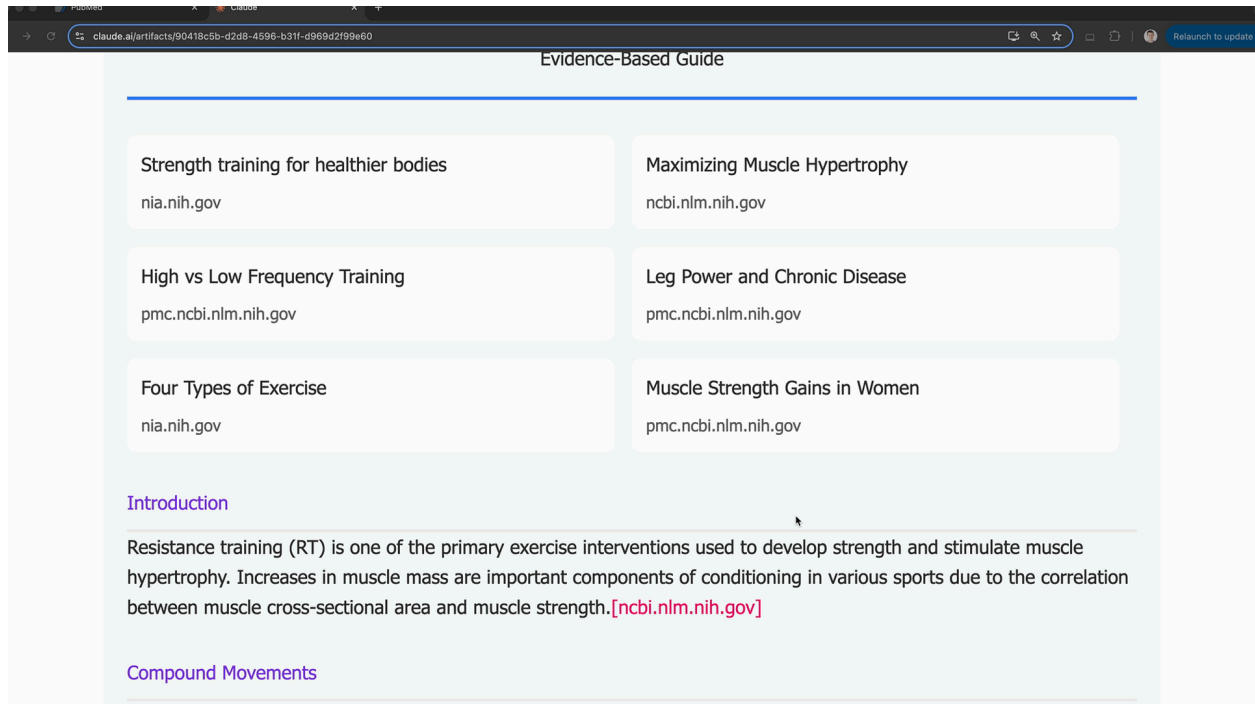
For example, when asking about medical or exercise advice, restricting to domains like PubMed (nih.gov) ensures you get evidence-based information rather than random blog content.



Rendering Search Results

The different block types in the response are designed for specific UI rendering:

- Render text blocks as regular content
- Display web search results as a list of sources at the top
- Show citations inline with the text, including the source domain, page title, URL, and quoted text



This structure helps users understand how Claude arrived at its answers and provides transparency about the sources being used. The citation format makes it clear which specific information came from which sources, building trust in the AI's responses.

Practical Usage

The web search tool works best for:

- Current events and recent developments
- Specialized information not in Claude's training data
- Fact-checking and finding authoritative sources
- Research tasks requiring up-to-date information

Simply include the schema in your tools array when making API calls, and Claude will automatically decide when a web search would help answer the user's question.